

习题大全

下面按 **Problem Set 1** → **Problem Set 2** 的顺序讲。每道题我都会先翻译原题，再讲解思路 and 答案。题目来自你上传的两份作业 PDF，参考答案来自你上传的两份 solution PDF。

Problem Set 1

第 1 题：带有并列偏好的稳定匹配

原题翻译

稳定匹配问题中，通常假设所有学生和医院都有一个**完全严格排序**的偏好列表，也就是说不存在并列偏好。

现在考虑一个变体：有 n 个学生集合 S 和 n 个医院集合 H 。每个学生和每个医院都对另一类对象进行排序，但现在允许出现**并列偏好**。

例如，当 $n = 4$ 时，一个医院可能认为：

- s_1 排第一；
- s_2 和 s_3 并列第二；
- s_4 排最后。

如果 s 在医院 h 的偏好表中排名严格高于 s' ，并且两者不是并列，则称 h 更喜欢 s 而不是 s' 。

有并列偏好后，稳定性可以有两种自然定义。

第 1(a) 问翻译

在一个完美匹配 M 中，若存在学生 s 和医院 h ，使得：

- s 严格更喜欢 h ，而不是自己在 M 中的匹配对象；
- h 也严格更喜欢 s ，而不是自己在 M 中的匹配对象；

那么称 (s, h) 构成一个 **strong instability**，即强不稳定对。

问题：是否总是存在一个完美匹配，使得它没有强不稳定对？

如果存在，请给出算法；如果不存在，请给出反例。

第 1(a) 问讲解

答案：总是存在。

核心思路是：

把所有并列偏好随便打破，变成严格偏好，然后运行 Gale-Shapley 稳定匹配算法。

例如，如果某个学生 s 对医院 h_i 和 h_j 无差异，我们就人为规定：

- 如果 $i < j$ ，则把 h_i 排在 h_j 前面；
- 如果 $j < i$ ，则把 h_j 排在 h_i 前面。

医院的偏好列表也用同样方法处理。

这样做之后，原来的“带并列偏好问题”被转化成了普通的“严格偏好稳定匹配问题”。对严格偏好问题，Gale-Shapley 算法一定能找到稳定匹配。

关键是：为什么这个匹配一定没有 **strong instability**？

假设我们打破并列后得到一个匹配 M 。如果 M 在原问题中存在强不稳定对 (s, h) ，那么说明：

- 学生 s 在原偏好中严格更喜欢 h ；
- 医院 h 在原偏好中也严格更喜欢 s 。

注意，这里的“严格更喜欢”不是并列，而是真的排名更高。因此，在我们打破并列之后，这种严格关系仍然保留，不会被改变。

所以 (s, h) 在打破并列后的严格偏好问题中也会是 blocking pair。

但 Gale-Shapley 算法得到的是严格偏好下的稳定匹配，不可能存在 blocking pair。

因此产生矛盾。

所以，任意打破并列后运行 Gale-Shapley 算法，得到的匹配一定没有 strong instability。

第 1(a) 问答案总结

算法如下：

1. 对所有学生和医院的偏好表，将并列项任意打破，得到严格排序；
2. 在严格排序上运行 Gale-Shapley 算法；
3. 输出得到的完美匹配。

正确性理由：

如果输出匹配存在 strong instability，那么这个不稳定对在打破并列后的严格偏好中仍然是不稳定对，矛盾。

所以一定能找到没有 strong instability 的完美匹配。

第 1(b) 问翻译

在一个完美匹配 M 中，若学生 s 的匹配对象是 h' ，医院 h 的匹配对象是 s' ，并且满足下面任一条件：

第一种情况：

- s 严格更喜欢 h 而不是 h' ；
- h 严格更喜欢 s 而不是 s' ，或者 h 对 s 和 s' 无差异；

第二种情况：

- h 严格更喜欢 s 而不是 s' ；
- s 严格更喜欢 h 而不是 h' ，或者 s 对 h 和 h' 无差异；

那么称 (s, h) 构成一个 **weak instability**，即弱不稳定对。

换句话说， (s, h) 这个配对要么被双方都严格偏好，要么被一方严格偏好，而另一方无差异。

问题：是否总是存在一个完美匹配，使得它没有 weak instability？

如果存在，请给出算法；如果不存在，请给出反例。

第 1(b) 问讲解

答案：**不一定存在。**

我们用一个 $n = 2$ 的反例说明。

有两个学生：

- s_1, s_2

有两个医院：

- h_1, h_2

偏好如下：

学生偏好：

- s_1 对 h_1 和 h_2 无差异；
- s_2 的偏好随便设，不重要。

医院偏好：

- $h_1 : s_1 > s_2$
- $h_2 : s_1 > s_2$

也就是说，两个医院都更喜欢 s_1 。

现在只有两个可能的完美匹配。

匹配一

$$M_1 = (s_1, h_1), (s_2, h_2)$$

此时考虑没有配在一起的组合 (s_1, h_2) 。

在 M_1 中：

- s_1 当前匹配 h_1 ；
- h_2 当前匹配 s_2 。

对于学生 s_1 ：

- s_1 对 h_1 和 h_2 无差异。

对于医院 h_2 ：

- h_2 严格更喜欢 s_1 ，而不是当前的 s_2 。

因此 (s_1, h_2) 满足 weak instability 的第二种情况：

- 医院一方严格更喜欢；
- 学生一方无差异。

所以 M_1 有 weak instability。

匹配二

$$M_2 = (s_1, h_2), (s_2, h_1)$$

此时考虑没有配在一起的组合 (s_1, h_1) 。

在 M_2 中：

- s_1 当前匹配 h_2 ;
- h_1 当前匹配 s_2 。

对于学生 s_1 :

- s_1 对 h_1 和 h_2 无差异。

对于医院 h_1 :

- h_1 严格更喜欢 s_1 , 而不是当前的 s_2 。

所以 (s_1, h_1) 也是 weak instability。

第 1(b) 问答案总结

两个可能的完美匹配都存在 weak instability, 因此并不总是存在没有 weak instability 的完美匹配。

这个反例说明:

strong instability 的要求更强, 必须双方都严格更喜欢, 所以总能避免;
weak instability 的要求更弱, 只要一方严格更喜欢、另一方不反对, 就可能构成不稳定, 因此不一定能完全避免。

第 2 题: 船舶停靠与稳定匹配

原题翻译

Peripatetic Shipping Lines Inc. 是一家航运公司, 拥有 n 艘船, 并服务于 n 个港口。

每艘船都有一个月度日程表。对于这个月的每一天, 日程表会说明这艘船当天:

- 正在访问哪个港口;
- 或者在海上。

可以假设这个月有 m 天, 并且 $m > n$ 。

每艘船在这个月中恰好访问每个港口一次。

为了安全, 公司有一个严格要求:

(†) 任何两艘船不能在同一天位于同一个港口。

现在公司想在这个月对所有船进行维护。方案如下:

对每艘船 S_i ，选择某一天，当它按照原日程抵达某个港口 P 时，就让它从那天开始一直停留在港口 P ，直到这个月结束，进行维护。

这意味着：

- S_i 在这个时间点之后不会继续访问原日程中剩下的港口；
- 但这是允许的。

所以，截断后的日程包括：

- 原日程从月初到某个停靠日为止；
- 之后一直停在某个港口。

问题是：

给定所有船的原始日程，能否为每艘船选择一个截断点，使得截断后的日程仍然满足：

任何两艘船在任何一天都不能位于同一个港口。

证明这样的截断方案一定存在，并给出算法。

题目直观理解

每艘船都必须选一个港口作为“最终维修港口”。

如果船 S_i 被安排最终停在港口 P_j ，那么它会按照原计划行驶到 P_j 那天，然后从那天开始一直留在 P_j 。

危险在于：

如果另一艘船后来还会经过 P_j ，那就会和已经停在 P_j 的船发生冲突。

所以我们要给每艘船分配一个不同的最终停靠港口，并保证不会产生后续冲突。

这道题的妙处在于：它可以转化成一个稳定匹配问题。

构造稳定匹配问题

我们构造一个二分匹配问题：

- 一边是 n 艘船；
- 另一边是 n 个港口。

然后定义双方偏好。

船的偏好

每艘船按照访问港口的时间先后排序。

如果船 S_i 先访问 P_a ，后访问 P_b ，那么规定：

S_i 更喜欢 P_a ，而不是 P_b 。

也就是说：

船更喜欢越早访问到的港口。

原因是：如果一艘船最终停在较晚的港口，那么它在停靠前会经过更多港口，因此更容易与别人冲突。

港口的偏好

每个港口按照船只访问它的时间倒序排序。

如果船 S_i 比船 S_j 更晚访问港口 P_k ，那么规定：

P_k 更喜欢 S_i ，而不是 S_j 。

也就是说：

港口更喜欢越晚访问自己的船。

原因是：如果某艘船较晚才到达某港口，那么让它停在那里更安全，因为较早访问该港口的船已经过去了，不会再回来。

算法

1. 对每艘船，按照其访问港口的时间，建立船对港口的偏好表；
 2. 对每个港口，按照各船访问该港口的时间倒序，建立港口对船的偏好表；
 3. 运行 Gale-Shapley 稳定匹配算法；
 4. 如果匹配结果中船 S_i 匹配到港口 P_j ，就让 S_i 在它原日程中访问 P_j 的那一天开始停在 P_j 。
-

正确性证明

我们要证明：稳定匹配对应的截断方案一定不会违反安全要求。

反证法。

假设根据稳定匹配得到的截断方案不安全。

那么存在两艘船 S_i, S_j 和一个港口 P_k ，使得：

- S_j 已经停在 P_k ；
- 之后某一天， S_i 又按照自己的截断前日程访问了 P_k 。

这就造成了两艘船同一天出现在同一港口，违反安全要求。

现在看这意味着什么。

因为 S_i 后来还访问了 P_k ，说明 S_i 在自己的最终停靠港口之前就已经访问过 P_k 。

所以按照船的偏好定义：

- S_i 更喜欢 P_k ，而不是自己实际匹配到的最终停靠港口。

再看港口 P_k 。

S_j 是匹配到 P_k 并停在那里的船，而 S_i 是后来才访问 P_k 的船。

按照港口偏好定义，港口更喜欢更晚访问自己的船。

所以：

- P_k 更喜欢 S_i ，而不是当前匹配的 S_j 。

于是 (S_i, P_k) 构成了一个 blocking pair：

- 船 S_i 更喜欢港口 P_k ；
- 港口 P_k 也更喜欢船 S_i 。

这与我们得到的是稳定匹配矛盾。

因此，截断方案一定是安全的。

第 2 题答案总结

这道题的核心是：

把“船最终停在哪个港口”的问题转化为“船和港口之间的稳定匹配问题”。

船偏好：越早访问的港口越好。

港口偏好：越晚访问自己的船越好。

运行 Gale-Shapley 后得到稳定匹配。将每艘船停在匹配到的港口，必然不会发生两船同港冲突。

第 3 题：Gale-Shapley 算法中的诚实性问题

原题翻译

本题研究稳定匹配问题中的 **truthfulness**，即诚实性问题，特别是 Gale-Shapley 算法中的诚实性。

基本问题是：

学生或医院是否可能通过谎报自己的偏好，从而得到更好的匹配对象？

更具体地说，假设每个参与者都有真实偏好。现在考虑某个医院 h 。

假设医院 h 真实地更喜欢学生 s ，而不是学生 s' ，但 s 和 s' 在它的偏好表中都比较靠后。

问题是：

如果医院 h 把 s 和 s' 的顺序交换，也就是谎称自己更喜欢 s' 而不是 s ，然后用这个虚假的偏好表运行 Gale-Shapley 算法，是否可能使医院 h 最终得到一个学生 s'' ，并且 h 真实地更喜欢 s'' ，超过 s 和 s' ？

你需要二选一：

1. 证明：无论偏好表如何，医院通过交换某一对学生的顺序都不可能改善自己的匹配结果；
2. 或者给出一个反例：某个医院通过交换偏好顺序确实能得到更好的匹配对象。

题目背景

Gale-Shapley 算法有一个重要性质：

对“主动提出申请的一方”是 strategy-proof 的，也就是说主动方诚实报偏好不会吃亏。
但对“被动接受的一方”不一定 strategy-proof。

在这道题的参考解答中，运行的是**学生向医院提出申请**的 Gale-Shapley 算法。此时：

- 学生是 proposing side，主动方；
- 医院是 accepting side，接受方。

所以医院有可能通过谎报偏好得到更好的结果。

反例设置

有三个学生：

- s_1, s_2, s_3

有三个医院：

- h_1, h_2, h_3

学生偏好如下：

学生	第一选择	第二选择	第三选择
s_1	h_3	h_1	h_2
s_2	h_1	h_3	h_2
s_3	h_3	h_1	h_2

医院真实偏好如下：

医院	第一选择	第二选择	第三选择
h_1	s_1	s_2	s_3
h_2	s_1	s_2	s_3
h_3	s_2	s_1	s_3

医院 h_3 的真实偏好是：

$$h_3 : s_2 > s_1 > s_3$$

也就是说， h_3 最喜欢 s_2 ，其次是 s_1 ，最不喜欢 s_3 。

现在让 h_3 说谎，把 s_1 和 s_3 的顺序交换，虚假偏好变成：

$$h'_3 : s_2 > s_3 > s_1$$

注意，它只是交换了 s_1 和 s_3 的顺序，但目标是最终得到更好的 s_2 。

情况一： h_3 诚实申报偏好

医院 h_3 的真实偏好为：

$$h_3 : s_2 > s_1 > s_3$$

运行学生提出申请的 Gale-Shapley 算法。

第一轮：

- s_1 向第一选择 h_3 申请；
- s_2 向第一选择 h_1 申请；
- s_3 向第一选择 h_3 申请。

医院处理申请：

- h_3 收到 s_1 和 s_3 ，根据真实偏好 $s_2 > s_1 > s_3$ ，它保留 s_1 ，拒绝 s_3 ；
- h_1 收到 s_2 ，暂时保留 s_2 。

于是：

- s_3 被拒绝。

第二轮：

- s_3 向第二选择 h_1 申请。

医院 h_1 当前保留 s_2 ，收到 s_3 后比较：

$$h_1 : s_1 > s_2 > s_3$$

所以 h_1 继续保留 s_2 ，拒绝 s_3 。

第三轮：

- s_3 向第三选择 h_2 申请；
- h_2 暂时保留 s_3 。

最终匹配为：

$$(s_1, h_3), (s_2, h_1), (s_3, h_2)$$

此时医院 h_3 得到的是 s_1 。

情况二： h_3 谎报偏好

现在 h_3 的虚假偏好为：

$$h'_3 : s_2 > s_3 > s_1$$

也就是它谎称自己更喜欢 s_3 而不是 s_1 。

重新运行学生提出申请的 Gale-Shapley 算法。

第一轮：

- s_1 向 h_3 申请；
- s_2 向 h_1 申请；
- s_3 向 h_3 申请。

医院处理申请：

- h_3 收到 s_1 和 s_3 ，根据虚假偏好 $s_2 > s_3 > s_1$ ，它保留 s_3 ，拒绝 s_1 ；

- h_1 保留 s_2 。

于是：

- s_1 被拒绝。

第二轮：

- s_1 向第二选择 h_1 申请。

医院 h_1 当前保留 s_2 ，收到 s_1 后比较：

$$h_1 : s_1 > s_2 > s_3$$

所以 h_1 改为保留 s_1 ，拒绝 s_2 。

于是：

- s_2 被拒绝。

第三轮：

- s_2 向第二选择 h_3 申请。

医院 h_3 当前保留 s_3 ，收到 s_2 后比较虚假偏好：

$$h'_3 : s_2 > s_3 > s_1$$

所以 h_3 改为保留 s_2 ，拒绝 s_3 。

第四轮：

- s_3 被 h_3 拒绝后，向第二选择 h_1 申请；
- h_1 当前保留 s_1 ，根据偏好 $s_1 > s_2 > s_3$ ，拒绝 s_3 ；
- s_3 再向第三选择 h_2 申请；
- h_2 保留 s_3 。

最终匹配为：

$$(s_1, h_1), (s_2, h_3), (s_3, h_2)$$

此时医院 h_3 得到的是 s_2 。

对比结果

诚实时：

- h_3 得到 s_1 。

说谎时：

- h_3 得到 s_2 。

而 h_3 的真实偏好是：

$$s_2 > s_1 > s_3$$

所以 h_3 通过说谎，从 s_1 改善到了 s_2 。

第 3 题答案总结

这说明：

在学生提出申请的 Gale-Shapley 算法中，医院作为接受方，可能通过谎报偏好获得更好的匹配对象。

所以第 3 题应选择：

给出反例，说明医院可以通过交换偏好列表中的一对学生顺序，从而改善最终匹配结果。

Problem Set 2

第 1 题：函数增长率排序

原题翻译

给定下面一组函数，请按照增长率从小到大的顺序排列它们。

要求是：如果函数 $g(n)$ 紧跟在函数 $f(n)$ 后面，那么应满足：

$$f(n) = O(g(n))$$

函数如下：

- $f_1(n) = n^{2.5}$
- $f_2(n) = \sqrt{2n}$
- $f_3(n) = n + 10$
- $f_4(n) = 10^n$
- $f_5(n) = 100^n$
- $f_6(n) = n^2 \log n$

这里注意：PDF 解析文本里 10^n 和 100^n 的上标可能被解析成了普通字符，但结合参考答案，这里应理解为指数函数 10^n 和 100^n 。

讲解

我们逐个判断增长阶。

$$f_2(n) = \sqrt{2n}$$

因为：

$$\sqrt{2n} = \sqrt{2}\sqrt{n}$$

所以：

$$f_2(n) = \Theta(n^{1/2})$$

它是根号级别，比线性函数慢。

$$f_3(n) = n + 10$$

常数 10 不影响渐近增长率，所以：

$$f_3(n) = \Theta(n)$$

显然：

$$n^{1/2} = O(n)$$

所以：

$f_2(n)$ 排在 $f_3(n)$ 前面。

$$f_6(n) = n^2 \log n$$

这是略高于二次函数的增长率。

因为：

$$n = O(n^2 \log n)$$

所以：

$f_3(n)$ 排在 $f_6(n)$ 前面。

$$f_1(n) = n^{2.5}$$

比较 $n^2 \log n$ 和 $n^{2.5}$ 。

看比值：

$$\frac{n^2 \log n}{n^{2.5}} = \frac{\log n}{n^{0.5}}$$

当 $n \rightarrow \infty$ 时:

$$\frac{\log n}{\sqrt{n}} \rightarrow 0$$

因为任何对数函数都比正幂函数增长慢。

所以:

$$n^2 \log n = O(n^{2.5})$$

因此:

$f_6(n)$ 排在 $f_1(n)$ 前面。

$$f_4(n) = 10^n$$

指数函数比任何多项式函数增长更快。

因此:

$$n^{2.5} = O(10^n)$$

所以:

$f_1(n)$ 排在 $f_4(n)$ 前面。

$$f_5(n) = 100^n$$

比较 10^n 和 100^n 。

因为:

$$100^n = (10^2)^n = 10^{2n}$$

所以:

$$\frac{10^n}{100^n} = \frac{10^n}{10^{2n}} = 10^{-n} \rightarrow 0$$

因此:

$$10^n = O(100^n)$$

所以:

$f_4(n)$ 排在 $f_5(n)$ 前面。

第 1 题答案

增长率从小到大为：

$$f_2, f_3, f_6, f_1, f_4, f_5$$

也就是：

$$\sqrt{2n}, n + 10, n^2 \log n, n^{2.5}, 10^n, 100^n$$

第 2 题：大 O 关系的判断

原题翻译

假设有两个函数 f 和 g ，并且已知：

$$f(n) = O(g(n))$$

对于下面每个命题，判断是真还是假，并给出证明或反例。

第 2(a) 问翻译

判断：

$$\log_2 f(n) = O(\log_2 g(n))$$

是否一定成立？

第 2(a) 问讲解

答案：在算法分析通常默认函数最终为正且 $g(n)$ 足够大时，这个命题为真。

因为已知：

$$f(n) = O(g(n))$$

根据大 O 定义，存在常数 $c > 0$ 和 n_0 ，使得当 $n \geq n_0$ 时：

$$f(n) \leq cg(n)$$

两边取 $\log_2$ ：

$$\log_2 f(n) \leq \log_2 (cg(n))$$

利用对数性质：

$$\log_2 (cg(n)) = \log_2 c + \log_2 g(n)$$

如果 $g(n) \geq 2$ ，那么：

$$\log_2 g(n) \geq 1$$

所以常数项 $\log_2 c$ 可以被 $\log_2 g(n)$ 吸收。

更具体地说，当 $g(n) \geq 2$ 时：

$$\log_2 c + \log_2 g(n) \leq (\log_2 c + 1) \log_2 g(n)$$

因此：

$$\log_2 f(n) \leq C \log_2 g(n)$$

其中 $C = \log_2 c + 1$ 是常数。

所以：

$$\log_2 f(n) = O(\log_2 g(n))$$

第 2(a) 问答案总结

在通常算法分析语境下，结论为：

True.

但要注意：严格数学上需要保证 $f(n), g(n)$ 最终为正，并且 $g(n) > 1$ ，否则对数可能无定义或右边为零。

第 2(b) 问翻译

判断：

$$2^{f(n)} = O(2^{g(n)})$$

是否一定成立？

第 2(b) 问讲解

答案：**False，不一定成立。**

原因是：大 O 允许 $f(n)$ 比 $g(n)$ 大一个常数倍，但这个常数倍放到指数上，会变成指数级差距。

取反例：

$$f(n) = 2n$$

$$g(n) = n$$

显然：

$$f(n) = 2n = O(n) = O(g(n))$$

因为 $f(n) \leq 2g(n)$ 。

但是看指数函数：

$$2^{f(n)} = 2^{2n} = 4^n$$

$$2^{g(n)} = 2^n$$

比较二者比值：

$$\frac{2^{f(n)}}{2^{g(n)}} = \frac{4^n}{2^n} = 2^n$$

当 $n \rightarrow \infty$ 时：

$$2^n \rightarrow \infty$$

所以不存在常数 C 使得：

$$4^n \leq C2^n$$

因此：

$$2^{f(n)} \neq O(2^{g(n)})$$

第 2(b) 问答案总结

命题为：

False.

反例：

$$f(n) = 2n, \quad g(n) = n$$

虽然 $f(n) = O(g(n))$ ，但 $2^{f(n)} = 4^n$ 不是 $O(2^n)$ 。

第 2(c) 问翻译

判断：

$$f(n)^2 = O(g(n)^2)$$

是否一定成立？

第 2(c) 问讲解

答案：True。

因为已知：

$$f(n) = O(g(n))$$

所以存在常数 $c > 0$ 和 n_0 ，使得当 $n \geq n_0$ 时：

$$f(n) \leq cg(n)$$

如果函数是非负的，平方后得到：

$$f(n)^2 \leq c^2g(n)^2$$

由于 c^2 仍然是常数，所以根据大 O 定义：

$$f(n)^2 = O(g(n)^2)$$

这里要注意，你上传的参考答案中这一小问写成了：

$$f(n)^2 \geq c^2g(n)^2$$

这个方向是笔误。正确方向应该是：

$$f(n)^2 \leq c^2g(n)^2$$

第 2(c) 问答案总结

命题为：

True.

理由是大 O 不等式可以平方，常数 c 会变成 c^2 ，仍然是常数。

第 3 题：数组区间和矩阵的运行时间分析

原题翻译

给定一个包含 n 个整数的数组：

$$A[1], A[2], \dots, A[n]$$

希望输出一个二维的 $n \times n$ 数组 B 。

当 $i < j$ 时， $B[i, j]$ 存储数组 A 从第 i 项到第 j 项的和：

$$B[i, j] = A[i] + A[i + 1] + \dots + A[j]$$

当 $i \geq j$ 时, $B[i, j]$ 的值不作要求, 输出什么都无所谓。

题目给出一个简单算法:

```
for i = 1, 2, ..., n do
  for j = i+1, i+2, ..., n do
    Add up array entries A[i] through A[j]
    Store the result in B[i, j]
  end for
end for
```

问题:

(a) 选择一个函数 $f(n)$, 给出这个算法运行时间的上界 $O(f(n))$ 。

(b) 对同一个函数 $f(n)$, 证明这个算法运行时间也是 $\Omega(f(n))$ 。这样就说明运行时间是 $\Theta(f(n))$ 。

(c) 上述算法虽然自然, 但有大量重复计算。请设计一个渐近意义上更快的算法, 即运行时间为 $O(g(n))$, 并且满足:

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$$

第 3(a) 问讲解: 上界

原算法中有两层循环。

外层:

$$i = 1, 2, \dots, n$$

最多执行 n 次。

内层:

$$j = i + 1, \dots, n$$

最多执行 n 次。

所以内外循环组合最多有 $O(n^2)$ 次。

但是每一次内层循环中, 它都要重新计算:

$$A[i] + A[i + 1] + \dots + A[j]$$

这个求和长度是:

$$j - i + 1$$

最坏情况下, 比如 $i = 1, j = n$, 需要加 n 个数, 所以单次求和最坏是 $O(n)$ 。

因此总时间上界为：

$$O(n^2) \cdot O(n) = O(n^3)$$

所以可以取：

$$f(n) = n^3$$

第 3(a) 问答案

原算法运行时间为：

$$O(n^3)$$

第 3(b) 问讲解：下界

要证明它不仅是 $O(n^3)$ ，还要证明它至少需要 $\Omega(n^3)$ 的时间。

也就是说，我们要找出足够多的循环迭代，并且这些迭代每次都要做足够长的求和。

考虑这些迭代：

$$i \leq \frac{n}{4}, \quad j \geq \frac{3n}{4}$$

这样的 (i, j) 有多少个？

- i 至少有大约 $\frac{n}{4}$ 种选择；
- j 至少有大约 $\frac{n}{4}$ 种选择。

所以这样的组合至少有：

$$\frac{n}{4} \cdot \frac{n}{4} = \frac{n^2}{16}$$

对于这些组合，每次求和长度至少是：

$$j - i + 1 \geq \frac{3n}{4} - \frac{n}{4} + 1 = \frac{n}{2} + 1$$

所以每次至少要做 $\Omega(n)$ 的工作。

于是这一部分迭代总工作量至少为：

$$\frac{n^2}{16} \cdot \frac{n}{2} = \frac{n^3}{32}$$

因此整个算法运行时间至少是：

$$\Omega(n^3)$$

第 3(b) 问答案

因为存在 $\Omega(n^2)$ 次迭代，每次至少需要 $\Omega(n)$ 时间，所以原算法运行时间为：

$$\Omega(n^3)$$

结合第 (a) 问：

$$T(n) = \Theta(n^3)$$

第 3(c) 问讲解：更快算法

原算法慢的原因是：

每次计算 $B[i, j]$ 时，都从头把 $A[i]$ 到 $A[j]$ 重新加一遍，造成大量重复计算。

例如：

$$B[1, 5] = A[1] + A[2] + A[3] + A[4] + A[5]$$

$$B[1, 6] = A[1] + A[2] + A[3] + A[4] + A[5] + A[6]$$

第二个结果其实只是在第一个结果基础上多加了 $A[6]$ 。

所以可以用前缀和。

前缀和算法

定义前缀和数组 P ：

$$P[0] = 0$$

$$P[k] = A[1] + A[2] + \cdots + A[k]$$

那么：

$$A[i] + A[i+1] + \cdots + A[j] = P[j] - P[i-1]$$

因此：

$$B[i, j] = P[j] - P[i-1]$$

算法伪代码

```
P[0] = 0
```

```
for k = 1 to n do
```

```
P[k] = P[k-1] + A[k]
end for

for i = 1 to n do
  for j = i+1 to n do
    B[i,j] = P[j] - P[i-1]
  end for
end for
```

运行时间分析

第一部分计算前缀和：

- 循环 n 次；
- 每次 $O(1)$ ；

所以时间是：

$O(n)$

第二部分填充 $B[i, j]$ ：

- 只需要填充 $i < j$ 的位置；
- 这样的元素数量是：

$$\frac{n(n-1)}{2} = O(n^2)$$

每个 $B[i, j]$ 只需一次减法，所以是 $O(1)$ 。

因此第二部分时间是：

$O(n^2)$

总时间为：

$$O(n) + O(n^2) = O(n^2)$$

为什么这个算法已经渐近最优？

因为输出数组 B 中本来就有 $\Theta(n^2)$ 个需要指定的元素，也就是所有 $i < j$ 的位置。

单是把这些结果写出来，就需要 $\Omega(n^2)$ 时间。

所以 $O(n^2)$ 已经是渐近最优的。

第 3(c) 问答案总结

更快算法是前缀和算法：

1. 先用 $O(n)$ 时间计算前缀和数组 P ；
2. 再用公式 $B[i, j] = P[j] - P[i - 1]$ 在 $O(1)$ 时间内计算每个区间和；
3. 总共有 $O(n^2)$ 个需要填充的位置。

因此总运行时间为：

$$O(n^2)$$

原算法是：

$$f(n) = n^3$$

新算法是：

$$g(n) = n^2$$

并且：

$$\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \lim_{n \rightarrow \infty} \frac{n^2}{n^3} = \lim_{n \rightarrow \infty} \frac{1}{n} = 0$$

所以新算法满足题目要求。

下面讲 **Problem Set 3** 和 **Problem Set 4**。我会按题号依次来：**先翻译原题，再讲思路、算法、伪代码、正确性证明和复杂度**。其中 Problem Set 3 第 1、2 题涉及图，DFS/BFS 树和拓扑序不唯一，我采用你上传的参考答案中的一种合法结果。

Problem Set 3

第 1 题：DFS 树和 BFS 树

原题翻译

给定 Figure 1 中的无向图。请画出这个图的一棵 **深度优先搜索树** (depth-first search tree, DFS tree) 和一棵 **广度优先搜索树** (breadth-first search tree, BFS tree)。构造这些树时，可以从任意节点开始。

一、这题考什么

这题考两个基本图搜索算法：

- **DFS**：一条路尽量往深走，走不动再回溯；

- **BFS**：从起点开始一层一层扩展，先访问距离起点最近的节点。

注意：DFS 树和 BFS 树都不唯一，因为：

1. 起点可以任意选；
2. 每个节点的邻居访问顺序也可以任意选。

所以只要你的树满足 DFS 或 BFS 的访问逻辑，就是合法答案。

二、采用起点 v_1

参考答案中从 v_1 开始构造。Figure 1 是无向图。根据图可以看出主要边包括：

- $v_1 - v_7$
- $v_1 - v_9$
- $v_9 - v_3$
- $v_9 - v_{10}$
- $v_3 - v_4$
- $v_4 - v_2$
- $v_2 - v_{10}$
- $v_{10} - v_8$
- $v_8 - v_6$
- $v_6 - v_4$
- $v_6 - v_5$

三、一棵合法 DFS 树

参考答案中的 DFS 树可以写成父子关系：

父节点	子节点
v_1	v_7
v_1	v_9
v_9	v_3
v_3	v_4
v_4	v_2
v_2	v_{10}
v_{10}	v_8
v_8	v_6
v_6	v_5

对应的 DFS 搜索顺序可以理解为：

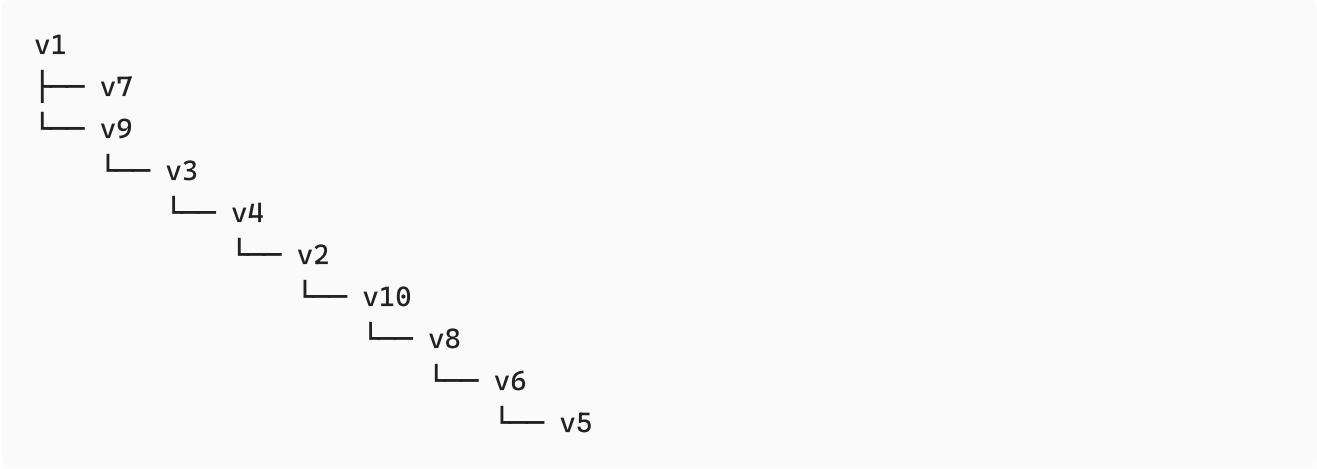
$v_1 \rightarrow v_7$

v_7 没有新的未访问邻居，回到 v_1 。

然后：

$v_1 \rightarrow v_9 \rightarrow v_3 \rightarrow v_4 \rightarrow v_2 \rightarrow v_{10} \rightarrow v_8 \rightarrow v_6 \rightarrow v_5$

所以 DFS 树可以表示为：



这符合 DFS 的特点：**能往深处走就一直往深处走。**

四、一棵合法 BFS 树

参考答案中的 BFS 树可以写成父子关系：

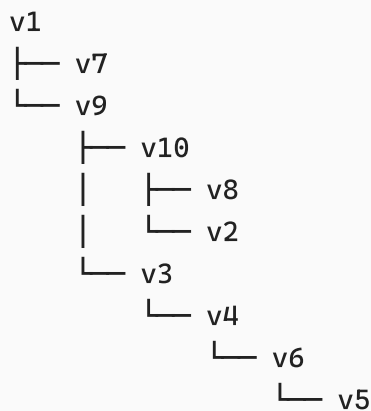
父节点	子节点
v_1	v_7
v_1	v_9
v_9	v_{10}
v_9	v_3
v_{10}	v_8
v_{10}	v_2
v_3	v_4
v_4	v_6
v_6	v_5

对应层次为：

- 第 0 层： v_1
- 第 1 层： v_7, v_9
- 第 2 层： v_{10}, v_3
- 第 3 层： v_8, v_2, v_4
- 第 4 层： v_6

- 第 5 层： v_5

BFS 树可以表示为：



这符合 BFS 的特点：先访问离起点距离为 1 的点，再访问距离为 2 的点，依次向外扩展。

五、DFS 和 BFS 的完整伪代码

DFS 伪代码

```
DFS(G, s):
    for each vertex u in G.V do
        visited[u] = false
        parent[u] = NIL
    end for

    DFS-VISIT(s)

    return parent

DFS-VISIT(u):
    visited[u] = true

    for each vertex x in Adj[u] do
        if visited[x] == false then
            parent[x] = u
            DFS-VISIT(x)
        end if
    end for
```

如果图不连通，需要对所有未访问节点继续调用 DFS：

```
DFS-ALL(G):
  for each vertex u in G.V do
    visited[u] = false
    parent[u] = NIL
  end for

  for each vertex u in G.V do
    if visited[u] == false then
      DFS-VISIT(u)
    end if
  end for

  return parent
```

BFS 伪代码

```
BFS(G, s):
  for each vertex u in G.V do
    visited[u] = false
    dist[u] = infinity
    parent[u] = NIL
  end for

  visited[s] = true
  dist[s] = 0
  parent[s] = NIL

  create an empty queue Q
  ENQUEUE(Q, s)

  while Q is not empty do
    u = DEQUEUE(Q)

    for each vertex x in Adj[u] do
      if visited[x] == false then
        visited[x] = true
        dist[x] = dist[u] + 1
        parent[x] = u
        ENQUEUE(Q, x)
      end if
    end for
  end while

  return parent, dist
```

第 2 题：拓扑排序

原题翻译

请给出 Figure 2 中有向图的一个 **拓扑排序** (topological ordering)。

一、什么是拓扑排序

对于一个有向图，如果能把所有节点排成一个线性序列，使得每条有向边 $u \rightarrow v$ 中， u 都排在 v 前面，那么这个序列就叫做拓扑排序。

拓扑排序只存在于 **有向无环图** (DAG, directed acyclic graph) 中。

二、一个合法拓扑序

参考答案给出的一个拓扑排序为：

$v_1, v_7, v_3, v_9, v_{10}, v_2, v_6, v_5, v_8, v_4$

也可以写成：

```
v1, v7, v3, v9, v10, v2, v6, v5, v8, v4
```

这个顺序的含义是：图中每条有向边都从左边指向右边。例如，如果有边 $v_3 \rightarrow v_9$ ，那么 v_3 必须排在 v_9 前面；如果有边 $v_{10} \rightarrow v_8$ ，那么 v_{10} 必须排在 v_8 前面。

三、拓扑排序的完整伪代码：Kahn 算法

Kahn 算法的核心思想是：

每次找一个入度为 0 的节点，把它加入答案，并删除它发出的边。

伪代码如下：

```
TOPOLOGICAL-SORT(G):  
  for each vertex  $u$  in  $G.V$  do  
    indegree[ $u$ ] = 0  
  end for  
  
  for each directed edge  $(u, v)$  in  $G.E$  do  
    indegree[ $v$ ] = indegree[ $v$ ] + 1  
  end for
```

```
create an empty queue Q

for each vertex u in G.V do
    if indegree[u] == 0 then
        ENQUEUE(Q, u)
    end if
end for

order = empty list

while Q is not empty do
    u = DEQUEUE(Q)
    append u to order

    for each vertex v in Adj[u] do
        indegree[v] = indegree[v] - 1
        if indegree[v] == 0 then
            ENQUEUE(Q, v)
        end if
    end for
end while

if length(order) < |G.V| then
    report "G has a directed cycle, so no topological ordering exists"
else
    return order
end if
```

四、复杂度

如果图有 n 个节点、 m 条边：

- 计算入度需要 $O(n + m)$ ；
- 每个节点入队出队一次；
- 每条边被处理一次；

所以总时间复杂度是：

$O(n + m)$

第 3 题：计算最短路径条数

原题翻译

给定一个无向图 $G = (V, E)$ ，并指定两个节点 v 和 w 。请给出一个算法，计算从 v 到 w 的最短路径条数。

注意：算法不应该列出所有路径，只需要返回最短路径的数量。

要求：如果图有 n 个节点、 m 条边，算法运行时间应为 $O(m + n)$ 。

一、题目重点

这题不是让你找一条最短路径，而是让你数：

从 v 到 w 一共有多少条最短路径。

不能暴力枚举所有路径，因为路径数量可能非常多，枚举会爆炸。

关键工具是 **BFS**。

在无权无向图中，BFS 从起点 v 出发，会按照距离分层：

- $L_0 = v$
- L_1 是所有距离 v 为 1 的点；
- L_2 是所有距离 v 为 2 的点；
- 依此类推。

一条从 v 到某个点 x 的路径是最短路径，当且仅当路径上节点层数每一步都增加 1。参考答案也利用了这个分层性质：最短路径只能从第 $j - 1$ 层走到第 j 层。

二、核心思想

设：

$count[x]$ 表示从 v 到 x 的最短路径条数。

初始化：

$$count[v] = 1$$

因为从 v 到自己有一条长度为 0 的路径。

然后 BFS 扫描边时：

如果通过 u 第一次发现 x ，说明：

$$dist[x] = dist[u] + 1$$

那么从 v 到 x 的最短路径条数暂时就是从 v 到 u 的最短路径条数：

$$count[x] = count[u]$$

如果之后又从另一个节点 u' 发现也可以到达 x ，并且满足：

$$dist[x] = dist[u'] + 1$$

说明 u' 也是 x 的前一层节点，也能贡献最短路径。

于是：

$$count[x] = count[x] + count[u']$$

三、完整伪代码

```
COUNT-SHORTEST-PATHS(G, v, w):
    for each vertex x in G.V do
        dist[x] = infinity
        count[x] = 0
    end for

    dist[v] = 0
    count[v] = 1

    create an empty queue Q
    ENQUEUE(Q, v)

    while Q is not empty do
        u = DEQUEUE(Q)

        for each vertex x in Adj[u] do

            if dist[x] == infinity then
                dist[x] = dist[u] + 1
                count[x] = count[u]
                ENQUEUE(Q, x)

            else if dist[x] == dist[u] + 1 then
                count[x] = count[x] + count[u]

            end if

        end for

    end while

    return count[w]
```

四、为什么这样是对的

证明思路是按 BFS 层数归纳。

第 0 层

$$L_0 = v_0$$

从 v 到 v 的最短路径只有一条空路径，所以：

$$\text{count}[v] = 1$$

正确。

第 1 层

如果 $x \in L_1$ ，说明 x 是 v 的邻居。

从 v 到 x 的最短路径长度是 1，路径只能是：

$$v \rightarrow x$$

所以每个第一层节点的最短路径条数就是它从 v 被连接到的方式数。对于普通无重边图，一般为 1。

第 j 层

设 $y \in L_j$ ，其中 $j > 1$ 。

任何从 v 到 y 的最短路径，最后一步必然来自某个第 $j - 1$ 层节点 x 。

原因是：

- 如果最后一步来自第 j 层，路径长度不会增加到正确层数；
- 如果来自第 $j + 1$ 层或更深层，更不可能是最短；
- 如果来自第 $j - 2$ 层或更浅层，那么 y 的距离应该更短，矛盾。

所以：

$$\text{count}[y] = \sum_{x \in L_{j-1}, (x,y) \in E} \text{count}[x]$$

这个公式正是伪代码中：

```
else if dist[x] == dist[u] + 1:
    count[x] = count[x] + count[u]
```

所做的事情。

因此算法正确。

五、复杂度分析

BFS 中：

- 每个节点最多入队一次；
- 每条边最多被检查两次，因为是无向图；

所以时间复杂度为：

$$O(n + m)$$

空间复杂度为：

$$O(n)$$

满足题目要求。

第 4 题：口述历史数据的一致性判断

原题翻译

你正在帮助一群民族志学者分析他们通过采访某个村庄成员收集到的口述历史数据。他们想了解过去两百年中曾经生活在这个村庄里的人。

通过采访，他们了解到一组已经去世的人：

$$P_1, P_2, \dots, P_n$$

他们还收集了一些关于这些人生活时间关系的事实。每个事实有两种形式之一：

1. 对某些 i, j ，人物 P_i 在人物 P_j 出生之前就已经去世；
2. 对某些 i, j ，人物 P_i 和人物 P_j 的生命时间至少有一部分重叠。

由于这些信息来自记忆和口口相传，他们不确定所有事实是否正确。所以他们希望你判断：这些数据是否至少在内部一致。

也就是说，是否可能存在一组出生日期和死亡日期，使得所有收集到的事实同时成立。

请给出一个高效算法：

- 如果可能，就为每个人给出一组出生和死亡日期，使所有事实成立；
- 如果不可能，就正确报告：这些事实不可能同时成立。

一、题目本质

这题本质是 约束一致性问题。

每个人 P_i 有两个时间点：

- b_i : birth date, 出生时间；
- d_i : death date, 死亡时间。

基本事实一定有：

$$b_i < d_i$$

因为一个人必须先出生，再死亡。

然后题目给出两类额外约束。

二、两类事实如何转成不等式

事实类型 1

“ P_i 在 P_j 出生之前已经去世。”

意思是：

$$d_i < b_j$$

也就是：

P_i 的死亡时间早于 P_j 的出生时间。

事实类型 2

“ P_i 和 P_j 的生命时间至少部分重叠。”

两个人生命区间分别为：

$$[b_i, d_i]$$

$$[b_j, d_j]$$

要让它们有重叠，必须满足：

$$b_i < d_j$$

并且：

$$b_j < d_i$$

解释一下：

- P_i 必须在 P_j 死亡之前已经出生；
- P_j 也必须在 P_i 死亡之前已经出生。

所以重叠关系可以拆成两个先后约束：

$$b_i < d_j$$

$$b_j < d_i$$

三、构造有向图

我们构造一个有向图 G 。

每个人 P_i 建两个节点：

- b_i ：表示 P_i 的出生时间；
- d_i ：表示 P_i 的死亡时间。

然后加边表示“时间先后关系”。

如果有边：

$$x \rightarrow y$$

就表示：

x 对应的时间必须早于 y 对应的时间。

必加边

对每个人 P_i ，一定有：

$$b_i < d_i$$

所以加边：

$$b_i \rightarrow d_i$$

对事实类型 1 加边

如果事实是：

P_i died before P_j was born

即：

$$d_i < b_j$$

加边：

$$d_i \rightarrow b_j$$

对事实类型 2 加边

如果事实是：

P_i 和 P_j 生命周期有重叠

即：

$$b_i < d_j$$

$$b_j < d_i$$

加边：

$$b_i \rightarrow d_j$$

$$b_j \rightarrow d_i$$

参考答案也是这样构图：每个人建立出生和死亡两个节点，先加 $b_i \rightarrow d_i$ ；若 P_i 死于 P_j 出生前，则加 $d_i \rightarrow b_j$ ；若两人生命重叠，则加 $b_i \rightarrow d_j$ 和 $b_j \rightarrow d_i$ 。

四、关键判断：图是否有环

构造完有向图之后，问题变成：

这个有向图能不能赋予每个节点一个时间，使得所有边都从较早时间指向较晚时间？

这正是拓扑排序问题。

如果图中有有向环，例如：

$$x_1 \rightarrow x_2 \rightarrow x_3 \rightarrow x_1$$

那么就要求：

$$x_1 < x_2 < x_3 < x_1$$

这是不可能的。

所以：

- 如果图有环，说明事实不一致；

- 如果图无环，可以做拓扑排序，并按照拓扑序依次赋予递增年份。

五、完整伪代码

设事实集合为 F ，每个事实有类型：

- $\text{BEFORE_DEATH_BIRTH}(i, j)$ ：表示 P_i 死于 P_j 出生前；
- $\text{OVERLAP}(i, j)$ ：表示 P_i 和 P_j 生命重叠。

```
CHECK-HISTORY-CONSISTENCY( $n, F$ ):
    create a directed graph  $G$ 

    for  $i = 1$  to  $n$  do
        create vertex  $b[i]$ 
        create vertex  $d[i]$ 
        add directed edge  $b[i] \rightarrow d[i]$  to  $G$ 
    end for

    for each fact in  $F$  do

        if fact is  $\text{BEFORE\_DEATH\_BIRTH}(i, j)$  then
            add directed edge  $d[i] \rightarrow b[j]$  to  $G$ 

        else if fact is  $\text{OVERLAP}(i, j)$  then
            add directed edge  $b[i] \rightarrow d[j]$  to  $G$ 
            add directed edge  $b[j] \rightarrow d[i]$  to  $G$ 

        end if

    end for

    order =  $\text{TOPOLOGICAL-SORT}(G)$ 

    if  $\text{TOPOLOGICAL-SORT}$  reports a cycle then
        return "inconsistent"
    end if

     $t = 1$ 
    for each vertex  $x$  in order do
        time[ $x$ ] =  $t$ 
         $t = t + 1$ 
    end for

    for  $i = 1$  to  $n$  do
        birth[ $i$ ] = time[ $b[i]$ ]
        death[ $i$ ] = time[ $d[i]$ ]
    end for
```

```
return "consistent", birth[1..n], death[1..n]
```

其中 `TOPOLOGICAL-SORT` 可以用第 2 题的 Kahn 算法。

六、正确性证明

如果算法报告 inconsistent，则确实不可能

算法报告 inconsistent，说明构造出的有向图存在有向环。

有向环意味着存在一串约束：

$$x_1 < x_2 < x_3 < \cdots < x_k < x_1$$

这要求 $x_1 < x_1$ ，显然不可能。

所以不存在任何出生死亡日期可以满足全部事实。

如果算法返回日期，则这些日期一定满足所有事实

如果图无环，则存在拓扑排序。

我们按照拓扑序给每个节点赋予递增时间：

- 排第一个的时间为 1；
- 排第二个的时间为 2；
- 依此类推。

因为拓扑序保证每条边 $x \rightarrow y$ 中， x 排在 y 前面，所以：

$$time[x] < time[y]$$

于是：

- 对每个人的边 $b_i \rightarrow d_i$ ，有 $birth_i < death_i$ ；
- 对事实类型 1 的边 $d_i \rightarrow b_j$ ，有 $death_i < birth_j$ ；
- 对事实类型 2 的边 $b_i \rightarrow d_j$ 和 $b_j \rightarrow d_i$ ，有 $birth_i < death_j$ 且 $birth_j < death_i$ ，所以两人生命区间有重叠。

因此返回的日期满足所有事实。

七、复杂度

图中有：

- $2n$ 个节点；
- 每个人一条基本边，共 n 条；
- 每条事实最多产生 2 条边。

设事实数量为 k ，则边数是 $O(n + k)$ 。

构图、检测有向环、拓扑排序都可以在线性时间完成。

总时间复杂度：

$O(n + k)$

Problem Set 4

Problem Set 4 包含三个贪心相关题目：燃料粒子变换、卡车装箱、字符串构造。题目原文来自你上传的 Problem Set 4。

第 1 题：把燃料粒子数变成 1 的最少操作数

原题翻译

量子反物质燃料以小颗粒形式存在。LAMBCHOP 的许多移动部件每次都需要一个燃料颗粒，但小黄人会把燃料颗粒成批倒进燃料入口。

你需要找出最有效的方法，把燃料颗粒数量调整到每次一个。

燃料控制机制允许三种操作：

1. 增加 1 个燃料颗粒；
2. 减少 1 个燃料颗粒；
3. 将整组燃料颗粒数除以 2。但由于把量子反物质颗粒切成两半会释放破坏性能量，所以只有当颗粒数为偶数时，安全系统才允许执行除以 2 的操作。

请写一个函数 `answer(n)`，输入是一个表示正整数 n 的字符串，返回把燃料颗粒数变成 1 所需的最少操作数。

要求证明算法正确。

例子：

$29 \rightarrow 28 \rightarrow 14 \rightarrow 7 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

一、题目本质

给定正整数 n ，允许：

- $n \rightarrow n + 1$
- $n \rightarrow n - 1$
- 如果 n 为偶数， $n \rightarrow n/2$

目标是用最少步数变成 1。

这是一个经典贪心问题。

二、贪心规则

最优策略是：

1. 如果 $n = 1$ ，结束；
2. 如果 n 是偶数，直接除以 2；
3. 如果 n 是奇数：
 - 如果 $n = 3$ ，做 $n = n - 1$ ；
 - 如果 $n \equiv 1 \pmod{4}$ ，做 $n = n - 1$ ；
 - 如果 $n \equiv 3 \pmod{4}$ 且 $n > 3$ ，做 $n = n + 1$ 。

换成程序条件：

```
if n is even:
    n = n / 2
else:
    if n == 3 or n mod 4 == 1:
        n = n - 1
    else:
        n = n + 1
```

三、为什么看 $n \bmod 4$

如果 n 是奇数，那么不能直接除以 2，必须先加 1 或减 1，让它变成偶数。

关键是：我们希望变成偶数之后能尽量多除几次 2。

情况 1： $n \equiv 1 \pmod{4}$

例如：

9, 13, 17, 29, ...

此时：

$$n - 1 \equiv 0 \pmod{4}$$

$$n + 1 \equiv 2 \pmod{4}$$

所以 $n - 1$ 至少能连续除以 2 两次，而 $n + 1$ 通常只能先除一次。

例如：

$$29 \rightarrow 28 \rightarrow 14 \rightarrow 7$$

而如果加：

$$29 \rightarrow 30 \rightarrow 15$$

显然 28 比 30 更有利。

所以 $n \equiv 1 \pmod{4}$ 时，应优先减 1。

情况 2： $n \equiv 3 \pmod{4}$

例如：

$$7, 11, 15, 31, \dots$$

此时：

$$n + 1 \equiv 0 \pmod{4}$$

$$n - 1 \equiv 2 \pmod{4}$$

所以 $n + 1$ 至少能连续除以 2 两次，而 $n - 1$ 通常只能先除一次。

例如：

$$7 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

比：

$$7 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 1$$

步数更少或不差。

所以 $n \equiv 3 \pmod{4}$ 时，一般应加 1。

特殊情况： $n = 3$

虽然 $3 \equiv 3 \pmod{4}$ ，但不能加 1。

比较：

- 减 1: $3 \rightarrow 2 \rightarrow 1$, 共 2 步;
- 加 1: $3 \rightarrow 4 \rightarrow 2 \rightarrow 1$, 共 3 步。

所以 $n = 3$ 要特殊处理, 选择减 1。

四、完整伪代码

```
ANSWER(n_string):
    n = integer value of n_string
    steps = 0

    while n != 1 do

        if n is even then
            n = n / 2

        else
            if n == 3 then
                n = n - 1

            else if n mod 4 == 1 then
                n = n - 1

            else
                n = n + 1

            end if
        end if

        steps = steps + 1

    end while

    return steps
```

五、例子: $n = 29$

29 是奇数, 且 $29 \equiv 1 \pmod{4}$, 所以减 1:

$29 \rightarrow 28$

28 是偶数:

$28 \rightarrow 14$

14 是偶数：

$14 \rightarrow 7$

7 是奇数，且 $7 \equiv 3 \pmod{4}$ ，并且 $7 > 3$ ，所以加 1：

$7 \rightarrow 8$

继续除以 2：

$8 \rightarrow 4 \rightarrow 2 \rightarrow 1$

总共 7 步。

六、正确性证明

参考答案也是按偶数和奇数分别证明：偶数时最优第一步是除以 2；奇数时根据模 4 选择 +1 或 -1，并单独处理 $n = 3$ 。

1. 偶数时，第一步必须除以 2

设 n 是偶数且 $n > 2$ 。

如果不除以 2，而是先做 +1 或 -1，那么 n 会变成奇数。奇数无法直接除以 2，下一步还得再做一次 +1 或 -1 才能变回偶数。

也就是说，如果偶数 n 不直接除以 2，至少会浪费额外操作。

而直接除以 2 可以立刻把规模减半，这是最有效的下降方式。

所以偶数情况下，最优策略第一步一定是：

$n \rightarrow n/2$

2. 奇数时，第一步只能是 +1 或 -1

如果 n 是奇数，除以 2 不可用。

所以第一步只能是：

$n \rightarrow n - 1$

或：

$n \rightarrow n + 1$

两者都会变成偶数。下一步按照上面的结论，都应该除以 2。

因此我们比较的是：

- $n \rightarrow n - 1 \rightarrow (n - 1)/2$
- $n \rightarrow n + 1 \rightarrow (n + 1)/2$

哪个后续更好。

3. 当 $n \equiv 1 \pmod{4}$ 时，减 1 不差于加 1

若 $n = 4k + 1$ ，则：

$$n - 1 = 4k$$

$$n + 1 = 4k + 2$$

减 1 之后得到 $4k$ ，能被 4 整除；加 1 之后得到 $4k + 2$ ，只能保证被 2 整除。

经过两步后：

- 减 1 路线到达 $2k$ ；
- 加 1 路线到达 $2k + 1$ 。

$2k$ 是偶数，通常更容易继续除以 2； $2k + 1$ 是奇数，还要再进行 +1 或 -1。

所以减 1 更优或至少不差。

4. 当 $n \equiv 3 \pmod{4}$ 且 $n > 3$ 时，加 1 不差于减 1

若 $n = 4k + 3$ ，则：

$$n + 1 = 4k + 4$$

$$n - 1 = 4k + 2$$

加 1 后得到能被 4 整除的数，减 1 后只能保证被 2 整除。

所以加 1 后更有利于连续除以 2。

唯一例外是 $n = 3$ ，因为：

$$3 \rightarrow 2 \rightarrow 1$$

比：

$$3 \rightarrow 4 \rightarrow 2 \rightarrow 1$$

少一步。

因此 $n = 3$ 时应减 1。

七、复杂度

每次操作都会大幅减少 n 的二进制长度，或者很快制造出可以除以 2 的情况。

所以操作次数是 $O(\log n)$ 量级。

如果 n 用字符串输入，转为大整数后处理，伪代码层面可认为循环次数为 $O(\log n)$ 。

第 2 题：卡车按顺序装箱的贪心最优性

原题翻译

你正在为一家货运公司提供咨询。这家公司每天在纽约和波士顿之间运输大量包裹，因此每天需要发出多辆卡车。

每辆卡车有固定的最大载重 W 。

包裹一个一个到达纽约站，第 i 个包裹重量为 w_i 。

货运站很小，所以任意时刻最多只能有一辆卡车在站内。

公司政策要求包裹必须按照到达顺序运输。否则，如果顾客看到比自己的包裹晚到的包裹先到波士顿，可能会不满。

目前公司使用一个简单贪心算法装车：

- 按包裹到达顺序装车；
- 如果下一个包裹装不进当前卡车，就发走当前卡车，换下一辆车。

公司担心这样是否用了太多卡车。他们想知道，如果有时提前发走一辆没装满的卡车，是否能让后面几辆车装得更好，从而减少卡车数量。

请证明：对于给定的一组包裹重量，当前使用的贪心算法实际上最小化了所需卡车数量。

证明应仿照区间调度问题中的分析方式，也就是找到一个衡量标准，证明贪心算法在这个标准下始终 “stays ahead”，即始终领先于所有其他方案。

一、题目本质

这题和普通 bin packing 不一样。

普通装箱问题可以重排物品，通常很难。

但这题有一个强约束：

包裹必须按到达顺序运输。

所以每辆卡车装的一定是一段连续包裹。

例如包裹序列是：

1, 2, 3, 4, 5, 6, 7

一种合法装法可能是：

- 第 1 辆：1, 2, 3
- 第 2 辆：4, 5
- 第 3 辆：6, 7

但不能是：

- 第 1 辆：1, 3, 5
- 第 2 辆：2, 4, 6

因为这破坏了顺序。

二、贪心算法

贪心策略是：

当前卡车能装就继续装；一旦下一个包裹装不下，就发车。

三、完整伪代码

输入：

- 包裹数量 k ；
- 重量数组 $w[1..k]$ ；
- 卡车容量 W 。

输出：

- 最少卡车数；
- 每辆车装哪些包裹。

```
GREEDY-TRUCK-PACKING( $w[1..k]$ ,  $W$ ):  
    trucks = empty list  
  
    current_truck = empty list  
    current_weight = 0  
  
    for  $i = 1$  to  $k$  do
```

```

    if current_weight + w[i] <= W then
        append package i to current_truck
        current_weight = current_weight + w[i]

    else
        append current_truck to trucks

        current_truck = empty list
        append package i to current_truck
        current_weight = w[i]

    end if

end for

if current_truck is not empty then
    append current_truck to trucks
end if

return trucks

```

如果只需要卡车数量：

```

GREEDY-TRUCK-COUNT( $w[1..k]$ ,  $W$ ):
    count = 1
    current_weight = 0

    for i = 1 to k do

        if current_weight + w[i] <= W then
            current_weight = current_weight + w[i]

        else
            count = count + 1
            current_weight = w[i]

        end if

    end for

    return count

```

四、证明思路：贪心始终领先

设一共有 k 个包裹，按顺序编号：

$1, 2, \dots, k$

设贪心算法用了 m 辆车。

定义：

$f(i)$ 表示贪心算法用第 i 辆车装完后，已经装到的最后一个包裹编号。

例如：

- $f(1) = 5$ 表示贪心第 1 辆车装到了第 5 个包裹；
- $f(2) = 9$ 表示贪心前 2 辆车装到了第 9 个包裹。

再考虑任意其他合法方案 H ，假设它用了 n 辆车。

定义：

$g(i)$ 表示方案 H 用第 i 辆车装完后，已经装到的最后一个包裹编号。

我们要证明：

$$f(i) \geq g(i)$$

对所有 i 都成立。

含义是：

用同样数量的卡车时，贪心算法装掉的包裹数不少于任何其他方案。

这就是 “stays ahead”。

参考答案也是用这个思想：设贪心第 i 辆车最后装到 $f(i)$ ，任意方案第 j 辆车最后装到 $g(j)$ ，然后证明 $f(i) \geq g(i)$ 。

五、归纳证明

基础情形： $i = 1$

贪心第一辆车会尽可能多装包裹，直到下一个包裹装不下。

因此，任何合法方案的第一辆车都不可能比贪心第一辆车装得更多。

所以：

$$f(1) \geq g(1)$$

归纳假设

假设对于第 j 辆车，有：

$$f(j) \geq g(j)$$

也就是说，用前 j 辆车时，贪心已经装到的位置不比方案 H 少。

证明第 $j + 1$ 辆车

贪心第 $j + 1$ 辆车从包裹 $f(j) + 1$ 开始装，一直装到 $f(j + 1)$ 。

由于贪心在 $f(j + 1) + 1$ 处停止，说明：

$$w_{f(j)+1} + w_{f(j)+2} + \cdots + w_{f(j+1)+1} > W$$

即如果再多装下一个包裹，就会超重。

而根据归纳假设：

$$g(j) \leq f(j)$$

所以方案 H 的第 $j + 1$ 辆车开始位置不会晚于贪心第 $j + 1$ 辆车。

如果方案 H 的第 $j + 1$ 辆车想装到超过 $f(j + 1)$ ，它就必须装下从某个不晚于 $f(j) + 1$ 的位置开始，到 $f(j + 1) + 1$ 的这一段包裹。

这段重量至少包含贪心已经无法再装下的那一段，因此会超过 W 。

所以方案 H 的第 $j + 1$ 辆车不可能装得比贪心更多。

因此：

$$f(j + 1) \geq g(j + 1)$$

归纳完成。

六、推出最优性

假设任意方案 H 用 n 辆车可以装完所有包裹，即：

$$g(n) = k$$

根据上面的结论：

$$f(n) \geq g(n) = k$$

说明贪心算法用 n 辆车也一定已经装完所有包裹。

所以贪心使用的卡车数 m 不可能大于 n 。

因此：

$$m \leq n$$

由于 H 是任意方案，贪心算法使用的卡车数不超过任何合法方案。

所以贪心算法最优。

七、复杂度

每个包裹只被检查一次。

时间复杂度：

$$O(k)$$

空间复杂度：

- 如果只返回卡车数量，是 $O(1)$ ；
 - 如果返回完整装车方案，是 $O(k)$ 。
-

第 3 题：由 T/F 模式和字符串生成字典序最小字符串

原题翻译

给定两个字符串：

- `str1`，长度为 n ，只由字符 'T' 和 'F' 构成；
- `str2`，长度为 m ，只由小写英文字母构成。

定义一个长度为 $n + m - 1$ 的字符串 `word`。如果对每个下标 $0 \leq i \leq n - 1$ ，它满足下面条件，则称 `word` 是由 `str1` 和 `str2` 生成的：

1. 如果 `str1[i] == 'T'`，那么 `word` 从下标 i 开始、长度为 m 的子串必须等于 `str2`。
即：

$$\text{word}[i \dots (i+m-1)] == \text{str2}$$

2. 如果 `str1[i] == 'F'`，那么 `word` 从下标 i 开始、长度为 m 的子串必须不等于 `str2`。
即：

$$\text{word}[i \dots (i+m-1)] != \text{str2}$$

请设计一个贪心算法，返回所有可能生成的字符串中字典序最小的那个。

一、题目理解

`word` 的长度是：

$$n + m - 1$$

因为 `str1` 的每个位置 i 都对应 `word` 中一个长度为 m 的窗口：

`word[i : i+m-1]`

也就是：

- `str1[0]` 检查 `word[0 : m-1]`
- `str1[1]` 检查 `word[1 : m]`
- ...
- `str1[n-1]` 检查 `word[n-1 : n+m-2]`

如果 `str1[i]='T'`，这个窗口必须等于 `str2`。

如果 `str1[i]='F'`，这个窗口必须不等于 `str2`。

目标是找字典序最小的 `word`。

二、贪心思想

分两步：

第一步：先满足所有 'T' 约束

如果 `str1[i]='T'`，那么 `word[i+j]` 必须等于 `str2[j]`，其中 $0 \leq j < m$ 。

这些位置是被强制决定的。

如果同一个位置被两个 'T' 约束要求成不同字符，就无解。

例如：

- 某个位置被要求是 'a'；
- 又被另一个约束要求是 'b'；

那么不可能存在合法 `word`。

第二步：未被强制的位置先填 'a'

为了让字典序最小，所有还没确定的位置都应该先填最小字符 'a'。

此时得到的是满足所有 'T' 约束的字典序最小候选串。

第三步：修复违反 'F' 的窗口

接下来检查所有 `str1[i]='F'` 的位置。

如果当前：

```
word[i : i+m-1] != str2
```

那么这个 'F' 条件已经满足，不用动。

如果当前：

```
word[i : i+m-1] == str2
```

那么必须修改这个窗口中的某个未被 'T' 强制的位置。

为了保持字典序尽量小，应该：

1. 选这个窗口中最靠右的可修改位置；
2. 把它从 'a' 改成 'b'。

为什么选最靠右？

因为字典序比较时，越靠前的位置越重要。改越靠右的位置，对整体字典序的影响越小。

为什么改成 'b'？

因为 'b' 是比 'a' 大的最小字符。既然当前窗口等于 `str2`，把某个 'a' 改成 'b' 就能破坏相等，同时增加尽量小。

参考答案也是这个思路：先处理所有 T，有冲突则无解；未赋值的位置作为 free bits，先填 'a'；若某个 F 窗口仍等于 `str2`，就把该范围内最后一个 free bit 改成 'b'。

三、完整伪代码

这里给出一个清晰版本。为了避免歧义，我把“free position”理解为：没有被 'T' 强制决定、目前仍可用于修复 'F' 的位置。

```
LEXICOGRAPHICALLY-SMALLEST-WORD(str1, str2):
    n = length(str1)
    m = length(str2)
    L = n + m - 1

    word[0..L-1] = array filled with '?'
    forced[0..L-1] = array filled with false
    free[0..L-1] = array filled with false
```

```

# Step 1: impose all T constraints
for i = 0 to n - 1 do
    if str1[i] == 'T' then
        for j = 0 to m - 1 do
            pos = i + j
            required_char = str2[j]

            if word[pos] != '?' and word[pos] != required_char then
                return "IMPOSSIBLE"
            end if

            word[pos] = required_char
            forced[pos] = true
        end for
    end if
end for

# Step 2: fill all unassigned positions with 'a'
for pos = 0 to L - 1 do
    if word[pos] == '?' then
        word[pos] = 'a'
        free[pos] = true
    end if
end for

# Step 3: fix all F constraints from left to right
for i = 0 to n - 1 do
    if str1[i] == 'F' then

        if SUBSTRING-EQUAL(word, i, str2) == true then

            p = -1

            # find the rightmost currently free position in this window
            for pos = i + m - 1 downto i do
                if free[pos] == true then
                    p = pos
                    break
                end if
            end for

            if p == -1 then
                return "IMPOSSIBLE"
            end if

            word[p] = 'b'
            free[p] = false

        end if
    end if
end for

```

```

        end if
    end for

    return word

SUBSTRING-EQUAL(word, start, str2):
    m = length(str2)

    for j = 0 to m - 1 do
        if word[start + j] != str2[j] then
            return false
        end if
    end for

    return true

```

四、例子理解

假设：

```
str1 = "TF"
```

```
str2 = "ab"
```

那么 word 长度为：

$$n + m - 1 = 2 + 2 - 1 = 3$$

设：

```
word = _ _ _
```

处理 T

str1[0]='T'，所以：

```
word[0:1] = "ab"
```

得到：

```
word = a b _
```

填空为 'a'

得到：

```
word = a b a
```

检查 F

`str1[1]='F'`，所以：

`word[1:2] != "ab"`

现在：

`word[1:2] = "ba"`

它不等于 `"ab"`，所以合法。

答案：

`"aba"`

五、正确性解释

1. 所有 'T' 约束必须优先满足

如果 `str1[i]='T'`，那么对应窗口必须等于 `str2`。这不是可以选择的，而是硬约束。

所以第一步直接把所有 'T' 对应的位置填好。

若出现冲突，说明某个位置被要求同时等于两个不同字符，因此无解。

2. 未强制位置先填 'a' 是字典序最小的起点

在所有满足 'T' 约束的字符串中，未强制位置没有限制。

为了使字典序最小，当然应该全部填最小字母 'a'。

所以第二步得到的是满足所有 'T' 约束的最小候选串。

3. 如果某个 'F' 窗口等于 `str2`，必须改其中某个非强制位置

如果 `str1[i]='F'`，但当前：

`word[i:i+m-1] == str2`

那么这个窗口违反要求。

要让它不等于 `str2`，至少要修改窗口中的一个字符。

但如果窗口里所有位置都是被 'T' 强制的，那就不能修改，因此无解。

如果存在可修改位置，就修改一个。

4. 为什么选最靠右的 free position

字典序比较时，从左往右看，第一个不同位置决定大小。

如果必须修改某个窗口，那么修改越靠右的位置，越能保持前缀不变，得到的整体字符串越小。

所以在当前必须破坏相等的窗口中，选最右边的可修改位置是贪心最优选择。

5. 为什么改成 'b'

因为这些 free 位置一开始填的是 'a'。

当前窗口等于 `str2`，说明这个 free 位置当前字符正好和 `str2` 对应字符相同。把它改为 'b' 可以破坏相等，并且 'b' 是比 'a' 大的最小字母。

所以这是最小幅度的修改。

6. 为什么不会破坏之前已经满足的 'F' 约束

这是本题证明中最技术的一点。

直观上，我们从左到右扫描 'F' 窗口。每次只在当前窗口中尽量靠右的位置做最小修改。参考答案中用两个引理说明：把某个 free bit 从 'a' 改成 'b' 后，不会让之前已经满足的 'F' 重新变成不满足。核心原因是，如果某个之前的窗口原本已经不等于 `str2`，只因为这个改动又变回等于 `str2`，那么 `str2` 在两个重叠窗口上会表现出某种周期性；而这会与当前窗口在修改前等于 `str2`、修改后被破坏这一事实冲突。

用更通俗的话说：

- 当前窗口等于 `str2`，所以必须修；
- 选最右 free 位置修，是对未来和过去影响最小的修法；
- 若这个修改会让某个旧的 'F' 窗口重新等于 `str2`，则说明两个窗口的重叠部分要求 `str2` 同时满足两个不兼容的对齐关系；
- 这与我们选择的 free block 和相等窗口结构矛盾。

因此，从左到右这样修复是安全的。

六、复杂度

上面给的是最直观伪代码。

- 处理所有 'T' 窗口需要 $O(nm)$;
- 检查所有 'F' 窗口, 每次比较长度 m , 也需要 $O(nm)$;
- 寻找右侧 free position 最坏也需要 $O(m)$, 整体仍是 $O(nm)$ 。

所以总时间复杂度为:

$O(nm)$

空间复杂度为:

$O(n + m)$

因为 word 长度是 $n + m - 1$ 。

总结

这两套题的核心知识点如下:

题目	核心方法
PS3 第 1 题	DFS 树、BFS 树
PS3 第 2 题	拓扑排序
PS3 第 3 题	BFS 分层 + 动态计数最短路径条数
PS3 第 4 题	时间约束建图 + 检测有向环 + 拓扑排序
PS4 第 1 题	数字变换贪心, 偶数除以 2, 奇数看 $n \bmod 4$
PS4 第 2 题	贪心装车, stay-ahead 归纳证明
PS4 第 3 题	先满足强制 T, 再贪心修复 F, 构造字典序最小字符串

下面继续讲 **Problem Set 5** 和 **Problem Set 6**。我会每题先翻译原题, 再给出算法思路、完整伪代码、正确性证明和复杂度分析。题目来自你上传的 Problem Set 5、6, 参考答案来自对应 solution 文件。

Problem Set 5

第 1 题: 矩阵迷宫中允许拆一堵墙的最短路径

一、原题翻译

你有空间站部分区域的地图。每张地图从一个监狱出口开始, 到一个逃生舱门结束。

地图用一个由 0 和 1 构成的矩阵表示:

- 0 表示可以通过的空地；
- 1 表示不能通过的墙。

监狱出口位于左上角 $(0, 0)$ ，逃生舱门位于右下角 $(w - 1, h - 1)$ 。

请写一个算法，求从监狱门到逃生舱的最短路径长度。作为改造计划的一部分，你允许拆除一堵墙。

二、题目本质

这是一个 **无权图最短路问题**。

普通迷宫最短路可以直接用 BFS，因为每走一步代价相同。

但这里多了一个条件：

最多可以拆掉一堵墙。

所以我们不能只记录“走到某个格子的最短距离”，还要记录：

- 走到这个格子时，是否已经拆过墙。

因此，每个格子需要拆成两个状态：

- 状态 0：到达该格子时还没有拆墙；
- 状态 1：到达该格子时已经拆过一堵墙。

也就是说，原来一个点 (r, c) ，现在变成两个状态：

$(r, c, 0)$ 和 $(r, c, 1)$ 。

这叫做 **状态扩展图** (state-expanded graph)。

三、核心想法

设矩阵为 M ，有 h 行、 w 列。

我们从起点状态：

$(0, 0, 0)$

开始 BFS。

对于当前位置 (r, c, b) ：

- $b = 0$ 表示还没拆过墙；

- $b = 1$ 表示已经拆过墙。

考虑相邻位置 (nr, nc) :

情况 1: 相邻位置是空地 0

可以直接走过去, 拆墙状态不变:

$$(r, c, b) \rightarrow (nr, nc, b)$$

情况 2: 相邻位置是墙 1

如果 $b = 0$, 说明还没拆过墙, 可以拆掉这堵墙并进入:

$$(r, c, 0) \rightarrow (nr, nc, 1)$$

如果 $b = 1$, 说明已经拆过墙, 不能再拆, 所以不能走。

四、完整伪代码

这里默认路径长度按“走了多少条边/多少步”计算, 即起点距离为 0。如果题目要求按“经过多少个格子”计算, 那么最终答案再加 1 即可。你上传的参考解答中把起点距离设为 0, 所以它采用的是“边数/步数”的定义。

算法: 允许拆一堵墙的最短路径 BFS

输入:

矩阵 M , 大小为 h 行 w 列

$M[r][c] \in \{0, 1\}$

起点为 $(0, 0)$

终点为 $(h-1, w-1)$

输出:

从起点到终点的最短路径长度;

如果不可达, 则返回 IMPOSSIBLE

步骤:

1. 对每个格子 (r, c) , 建立两个距离:

$\text{dist}[r][c][0]$ = 从起点到 (r, c) , 且没有拆过墙的最短距离

$\text{dist}[r][c][1]$ = 从起点到 (r, c) , 且已经拆过一堵墙的最短距离

2. 初始化:

对所有 r, c, b :

$\text{dist}[r][c][b] = +\infty$

$\text{dist}[0][0][0] = 0$

3. 建立一个队列 Q :

将状态 $(0,0,0)$ 入队

4. 当 Q 非空时, 重复执行:

取出队首状态 (r,c,b)

如果 (r,c) 是终点:

可以直接返回 $\text{dist}[r][c][b]$

因为 BFS 第一次到达终点时就是最短距离

对四个方向进行扩展:

上: $(-1,0)$

下: $(1,0)$

左: $(0,-1)$

右: $(0,1)$

对每个相邻格子 (nr,nc) :

如果 (nr,nc) 越界:

跳过

如果 $M[nr][nc] == 0$:

如果 $\text{dist}[nr][nc][b] == +\infty$:

$\text{dist}[nr][nc][b] = \text{dist}[r][c][b] + 1$

将 (nr,nc,b) 入队

如果 $M[nr][nc] == 1$:

如果 $b == 0$ 且 $\text{dist}[nr][nc][1] == +\infty$:

$\text{dist}[nr][nc][1] = \text{dist}[r][c][0] + 1$

将 $(nr,nc,1)$ 入队

5. 如果 BFS 结束仍未到达终点:

返回 IMPOSSIBLE

五、为什么这个算法正确

证明的核心是:

我们把“最多拆一堵墙”的问题, 转化成了普通图上的 BFS 最短路问题。

原问题中, 一个人走到同一个格子, 可能有两种完全不同的情况:

- 还没拆过墙;
- 已经拆过墙。

这两种状态的未来选择不同:

- 没拆过墙的人以后还能拆；
- 已拆过墙的人以后不能拆。

所以它们必须被视为两个不同状态。

1. BFS 找到的是扩展状态图中的最短路

在扩展状态图里，每次移动都对应一条边，边权都是 1。

BFS 在无权图中有标准性质：

第一次访问到某个状态时，得到的就是从起点到该状态的最短距离。

因此，BFS 得到的 $dist[r][c][b]$ 是从起点到状态 (r, c, b) 的最短距离。

2. 扩展图中的合法路径等价于原迷宫中的合法路径

原迷宫中的任意合法路径，最多拆一堵墙。

如果路径没有拆墙，那么它在扩展图中一直处于状态 $b = 0$ 。

如果路径拆了一堵墙，那么在走进那堵墙时，状态从 0 变为 1，之后一直保持 1。

所以原问题的任意合法路径，都对应扩展图中的一条路径。

反过来，扩展图中的任意路径也不可能拆超过一堵墙，因为状态一旦从 0 变成 1，就不能再次进入墙。

所以两者等价。

3. 终点答案

终点有两个状态：

- $(h - 1, w - 1, 0)$ ：到终点时没有拆墙；
- $(h - 1, w - 1, 1)$ ：到终点时已经拆过墙。

答案应为：

$\min dist[h - 1][w - 1][0], dist[h - 1][w - 1][1]$

如果在 BFS 中第一次遇到终点就返回，也是正确的，因为 BFS 按距离从小到大扩展。

六、复杂度分析

矩阵有 hw 个格子。

每个格子有 2 个状态，所以状态数是：

$$2hw = O(hw)$$

每个状态最多检查 4 个方向，所以边数也是：

$$O(hw)$$

因此：

- 时间复杂度： $O(hw)$ ；
 - 空间复杂度： $O(hw)$ 。
-

第 2 题：最小瓶颈路径问题

一、原题翻译

给定一个有向图 $G = (V, E)$ ，每条边都有非负长度，并给定一个起点 s 。

定义一条路径的 **瓶颈值** (bottleneck) 为这条路径上所有边长度中的最大值，而不是边长之和。

请修改 Dijkstra 算法，使得它能对每个顶点 $v \in V$ ，计算从 s 到 v 的所有路径中，瓶颈值最小的那条路径的瓶颈值。

要求算法运行时间为 $O(mn)$ ，其中 m 是边数， n 是点数。并且必须证明算法正确。

额外问题：能否用 Union-Find 设计一个运行时间为 $O(\log m(n + m\alpha(n)))$ 的算法？这部分没有分数。

二、普通最短路和瓶颈最短路的区别

普通最短路中，一条路径的长度是边权之和：

$$\ell(P) = \sum_{e \in P} \ell(e)$$

而这里的瓶颈路径中，一条路径的代价是最大边权：

$$b(P) = \max_{e \in P} \ell(e)$$

例如路径边权为：

2, 5, 3, 1

普通路径长度是：

$$2 + 5 + 3 + 1 = 11$$

瓶颈值是：

$$\max 2, 5, 3, 1 = 5$$

所以这题要最小化的是“最坏的一条边”。

三、修改 Dijkstra 的核心

在普通 Dijkstra 中，松弛边 (u, v) 时更新：

$$dist[v] = \min dist[v], dist[u] + \ell(u, v)$$

在瓶颈路径中，如果从 s 到 u 的最小瓶颈值是 $b[u]$ ，再走边 (u, v) ，那么这条新路径的瓶颈值是：

$$\max b[u], \ell(u, v)$$

所以更新规则改为：

$$b[v] = \min b[v], \max(b[u], \ell(u, v))$$

选择下一个点时，选择当前估计瓶颈值最小的未确定点。

参考答案也是这个思路：维护已探索集合 S ，每次选择一个未探索节点 v ，使得从 S 经一条边到 v 得到的候选瓶颈值 $\min_{(u,v)} \max b(u), \ell(u, v)$ 最小。

四、完整伪代码： $O(mn)$ 版本

这里不用堆，直接每轮扫描所有边，因此满足题目要求的 $O(mn)$ 。

算法：最小瓶颈路径 Dijkstra

输入：

有向图 $G=(V, E)$

每条边 $e=(u, v)$ 有非负长度 $\ell(e)$

起点 s

输出：

对每个顶点 v ，输出 $b[v]$

其中 $b[v]$ 是从 s 到 v 的最小瓶颈值

步骤：

1. 初始化：


```
对每个顶点  $v \in V$ :  
     $b[v] = +\infty$   
     $\text{parent}[v] = \text{NIL}$   
     $\text{visited}[v] = \text{false}$ 
```

```
 $b[s] = 0$ 
```

2. 重复 n 次:

在所有未确定的顶点 x 中, 选择 $b[x]$ 最小的顶点 u

如果 $b[u] == +\infty$:
 说明剩余顶点从 s 不可达, 可以停止

```
 $\text{visited}[u] = \text{true}$ 
```

对每条从 u 出发的边 $e=(u,v)$:

如果 $\text{visited}[v] == \text{false}$:

```
         $\text{candidate} = \max(b[u], l(e))$ 
```

如果 $\text{candidate} < b[v]$:
 $b[v] = \text{candidate}$
 $\text{parent}[v] = u$

3. 返回所有 $b[v]$ 和 $\text{parent}[v]$

如果严格按照参考答案中“每轮扫描所有跨越 S 到 $V - S$ 的边”的写法, 也可以写成:

算法: 按集合 S 扩张的最小瓶颈路径

1. $S = \{s\}$

2. $b[s] = 0$

3. 对所有 $v \neq s$:

```
     $b[v] = +\infty$   
     $\text{parent}[v] = \text{NIL}$ 
```

4. 当 $S \neq V$ 时:

```
     $\text{best\_value} = +\infty$   
     $\text{best\_vertex} = \text{NIL}$   
     $\text{best\_parent} = \text{NIL}$ 
```

对每条边 $e=(u,v) \in E$:

如果 $u \in S$ 且 $v \notin S$:

```
candidate = max(b[u], l(e))
```

```
    如果 candidate < best_value:  
        best_value = candidate  
        best_vertex = v  
        best_parent = u
```

```
    如果 best_vertex == NIL:  
        说明剩余顶点不可达，停止
```

```
    将 best_vertex 加入 S  
    b[best_vertex] = best_value  
    parent[best_vertex] = best_parent
```

5. 返回 b

这个版本更接近题目要求的 $O(mn)$ 分析。

五、正确性证明

证明方式和 Dijkstra 类似，使用归纳法。

设 S 是已经确定最小瓶颈值的点集。

我们要证明如下不变式：

每次循环开始时，对于所有 $u \in S$ ， $b[u]$ 已经等于从 s 到 u 的最小可能瓶颈值。

1. 初始情况

一开始：

$S = s$

从 s 到 s 的路径不需要经过任何边，瓶颈值可以定义为 0。

所以：

$b[s] = 0$

不变式成立。

2. 归纳假设

假设当前 S 中所有点的 b 值都已经正确。

算法下一步选择一个点 $v \notin S$ ，使得：

$$b'(v) = \min_{(u,v), u \in S} \max b[u], \ell(u, v)$$

在所有候选点中最小。

设算法选择的是边 (u, v) ，于是：

$$b[v] = \max b[u], \ell(u, v)$$

我们要证明这个值就是从 s 到 v 的最小瓶颈值。

3. 反证或割性质证明

考虑任意一条从 s 到 v 的路径 P 。

因为 $s \in S$ ，而 $v \notin S$ ，这条路径一定会从 S 第一次走到 $V - S$ 。

设第一次跨出 S 的边是：

$$(x, y)$$

其中：

$$x \in S, \quad y \notin S$$

路径 P 从 s 到 y 的这部分，至少要包含从 s 到 x 的路径以及边 (x, y) 。

根据归纳假设，从 s 到 x 的最小瓶颈值是 $b[x]$ 。

所以路径 P 的瓶颈值至少为：

$$\max b[x], \ell(x, y)$$

而算法选择 v 时，是在所有这种跨边中选候选瓶颈值最小的，所以：

$$b[v] \leq \max b[x], \ell(x, y) \leq b(P)$$

因此，任意 s 到 v 的路径 P 的瓶颈值都至少是 $b[v]$ 。

另一方面，算法确实通过 s 到 u 的最优瓶颈路径再加上边 (u, v) 构造出了一条瓶颈值为 $b[v]$ 的路径。

所以 $b[v]$ 正是最小瓶颈值。

六、复杂度分析

如果采用“每轮扫描所有边”的版本：

- 最多加入 n 个顶点；
- 每一轮扫描所有 m 条边；

所以时间复杂度为：

$$O(mn)$$

空间复杂度为：

$$O(n + m)$$

Problem Set 6

第 1 题：递推式上界证明

一、原题翻译

假设：

$$T(n) \leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n$$

并且：

$$T(1) = 0$$

证明下面两个结论：

1. 当 $\alpha + \beta < 1$ 时， $T(n) = O(n)$ ；
 2. 当 $\alpha + \beta = 1$ 时， $T(n) = O(n \log n)$ 。
-

二、这题在考什么

这个递推式描述的是一种分治算法：

- 原问题规模是 n ；
- 分成两个子问题，规模大约是 αn 和 βn ；
- 合并或其他额外工作量是 n 。

关键看子问题规模之和：

- 如果 $\alpha + \beta < 1$ ，下一层总规模变小得更快，所以总工作量是线性的；
- 如果 $\alpha + \beta = 1$ ，每一层总规模大约还是 n ，但有 $O(\log n)$ 层，所以总工作量是 $O(n \log n)$ 。

参考答案使用归纳法证明这两部分。需要注意，参考答案第 1(b) 处应为证明 $T(n) \leq cn \log n$ ，不是 $T(n) \leq c \log n$ ；后续推导实际使用的也是 $cn \log n$ 。

三、第 1(a) 问：当 $\alpha + \beta < 1$ 时证明 $T(n) = O(n)$

证明思路

我们用数学归纳法证明：

$$T(n) \leq cn$$

其中 c 是一个足够大的常数。

由递推式：

$$T(n) \leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n$$

如果归纳假设成立，则：

$$T(\lfloor \alpha n \rfloor) \leq c \lfloor \alpha n \rfloor \leq c \alpha n$$

$$T(\lfloor \beta n \rfloor) \leq c \lfloor \beta n \rfloor \leq c \beta n$$

所以：

$$T(n) \leq c \alpha n + c \beta n + n$$

即：

$$T(n) \leq [c(\alpha + \beta) + 1]n$$

为了让它不超过 cn ，需要：

$$c(\alpha + \beta) + 1 \leq c$$

等价于：

$$1 \leq c(1 - \alpha - \beta)$$

所以只要取：

$$c \geq \frac{1}{1 - \alpha - \beta}$$

即可。

因为 $\alpha + \beta < 1$ ，所以分母为正，可以取到这样的常数。

第 1(a) 问证明整理

取常数 c ，使得：

$$c \geq \frac{1}{1 - \alpha - \beta}$$

对 n 作归纳。

基础情况： $n = 1$ 时， $T(1) = 0 \leq c$ ，成立。

归纳步骤：

假设对所有小于 n 的规模都成立，则：

$$T(n) \leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n$$

$$\leq c\lfloor \alpha n \rfloor + c\lfloor \beta n \rfloor + n$$

$$\leq c\alpha n + c\beta n + n$$

$$= [c(\alpha + \beta) + 1]n$$

由于 $c \geq \frac{1}{1-\alpha-\beta}$ ，所以：

$$c(\alpha + \beta) + 1 \leq c$$

因此：

$$T(n) \leq cn$$

所以：

$$T(n) = O(n)$$

四、第 1(b) 问：当 $\alpha + \beta = 1$ 时证明 $T(n) = O(n \log n)$

证明思路

这次猜测：

$$T(n) \leq cn \log n$$

代入递推式。

由归纳假设：

$$T(\lfloor \alpha n \rfloor) \leq c\alpha n \log(\alpha n)$$

$$T(\lfloor \beta n \rfloor) \leq c\beta n \log(\beta n)$$

暂时忽略取整，因为取整只会让规模更小，不影响大 O 上界。

于是：

$$T(n) \leq c\alpha n \log(\alpha n) + c\beta n \log(\beta n) + n$$

展开：

$$\log(\alpha n) = \log n + \log \alpha$$

$$\log(\beta n) = \log n + \log \beta$$

所以：

$$T(n) \leq c\alpha n(\log n + \log \alpha) + c\beta n(\log n + \log \beta) + n$$

整理：

$$T(n) \leq c(\alpha + \beta)n \log n + cn(\alpha \log \alpha + \beta \log \beta) + n$$

因为 $\alpha + \beta = 1$ ，所以：

$$T(n) \leq cn \log n + cn(\alpha \log \alpha + \beta \log \beta) + n$$

注意当 $0 < \alpha, \beta < 1$ 时：

$$\log \alpha < 0, \quad \log \beta < 0$$

因此：

$$\alpha \log \alpha + \beta \log \beta < 0$$

这个负项可以抵消最后的 $+n$ 。只要 c 足够大，就有：

$$cn(\alpha \log \alpha + \beta \log \beta) + n \leq 0$$

于是：

$$T(n) \leq cn \log n$$

所以：

$$T(n) = O(n \log n)$$

五、第 1(b) 问证明整理

令：

$$H = -(\alpha \log \alpha + \beta \log \beta)$$

因为 $0 < \alpha, \beta < 1$ 且 $\alpha + \beta = 1$ ，所以 $H > 0$ 。

取：

$$c \geq \frac{1}{H}$$

归纳假设：

$$T(k) \leq ck \log k$$

对所有 $k < n$ 成立。

则：

$$T(n) \leq T(\lfloor \alpha n \rfloor) + T(\lfloor \beta n \rfloor) + n$$

$$\begin{aligned}
&\leq c\alpha n \log(\alpha n) + c\beta n \log(\beta n) + n \\
&= c\alpha n(\log n + \log \alpha) + c\beta n(\log n + \log \beta) + n \\
&= c(\alpha + \beta)n \log n + cn(\alpha \log \alpha + \beta \log \beta) + n \\
&= cn \log n - cnH + n
\end{aligned}$$

由于 $cH \geq 1$ ，所以：

$$-cnH + n \leq 0$$

因此：

$$T(n) \leq cn \log n$$

所以：

$$T(n) = O(n \log n)$$

第 2 题：Median-of-Medians 分组大小为 7 和 3 的运行时间

一、原题翻译

假设我们把 n 个元素分成 $\lfloor n/r \rfloor$ 组，每组有 r 个元素，并且使用这些组的中位数的中位数作为 pivot。

1. 当 $r = 7$ 时，证明最坏情况运行时间是 $O(n)$ ；
2. 当 $r = 3$ 时，解释为什么最坏情况运行时间不是 $O(n)$ 。

二、背景：Median-of-Medians 是什么

这是选择算法 SELECT 的核心。

目标是在线性时间内找第 k 小元素。

算法大致是：

1. 把 n 个元素分组；
2. 每组内部排序，找出每组中位数；
3. 递归地找这些中位数的中位数；
4. 用这个“中位数的中位数”作为 pivot；
5. 根据 pivot 分区；
6. 只在包含第 k 小元素的一侧递归。

关键在于：

pivot 不能太偏，否则递归规模太大。

第 2(a) 问： $r = 7$ 时证明 $O(n)$

一、分组为 7 时的结构

每组 7 个元素，排序后中位数是第 4 个。

一组中：

- 有 3 个元素小于组内中位数；
- 有 3 个元素大于组内中位数；
- 中位数本身在中间。

设所有组中位数的中位数为 p ，也就是 pivot。

至少有一半组的中位数大于等于 p 。

对于这些组，每组至少有 4 个元素大于等于 p ：

- 组内中位数；
- 中位数右边的 3 个元素。

所以至少有大约：

$$4 \cdot \frac{1}{2} \cdot \frac{n}{7} = \frac{2n}{7}$$

个元素大于等于 p 。

同理，至少有 $\frac{2n}{7}$ 个元素小于等于 p 。

因此分区后，较大的一侧最多有：

$$n - \frac{2n}{7} = \frac{5n}{7}$$

个元素。

严格考虑取整时，参考答案写成递推式：

$$T(n) \leq T(\lfloor n/7 \rfloor) + T(n - 4\lfloor n/14 \rfloor) + O(n)$$

其中：

- $T(\lfloor n/7 \rfloor)$ 是递归找中位数的中位数；
 - $T(n - 4\lfloor n/14 \rfloor)$ 是分区后最坏一侧的递归；
 - $O(n)$ 是分组、组内排序、分区的线性代价。
-

二、为什么递推式是线性的

粗略看：

$$T(n) \leq T(n/7) + T(5n/7) + O(n)$$

两个递归子问题规模之和是：

$$\frac{n}{7} + \frac{5n}{7} = \frac{6n}{7} < n$$

这正好符合上一题第 1(a) 的情况：

如果子问题规模之和小于 n ，总运行时间就是 $O(n)$ 。

三、完整伪代码

算法：SELECT-7(A, k)

输入：

数组 A，长度为 n

正整数 k，表示要找第 k 小元素

输出：

A 中第 k 小的元素

步骤：

1. 如果 n 很小：

直接排序 A

返回排序后第 k 个元素

2. 将 A 分成若干组：

每组 7 个元素

最后一组不足 7 个时可以单独处理

3. 对每一组进行排序：

找出该组的中位数

将所有组中位数组成数组 M

4. 递归求 M 的中位数：

`pivot = SELECT-7(M, ceil(|M|/2))`

5. 按 pivot 对 A 分区：

`L = {x ∈ A : x < pivot}`

`E = {x ∈ A : x = pivot}`

`R = {x ∈ A : x > pivot}`

6. 如果 $k \leq |L|$ ：

返回 `SELECT-7(L, k)`

否则如果 $k \leq |L| + |E|$:

返回 `pivot`

否则:

返回 `SELECT-7(R, k - |L| - |E|)`

四、正确性说明

这个算法和普通快速选择一样:

- `pivot` 把数组分成小于、等于、大于三部分;
- 第 k 小要么在左边, 要么就是 `pivot`, 要么在右边;
- 每次递归只递归到可能包含答案的一侧。

所以算法一定返回第 k 小元素。

关键不是正确性, 而是 `pivot` 质量足够好, 从而保证线性时间。

五、时间复杂度证明

分组、组内排序、找中位数、分区都是 $O(n)$ 。

递归包括:

- 在 $n/7$ 个中位数中找中位数;
- 在最大不超过 $5n/7$ 的一侧继续选择。

所以:

$$T(n) \leq T(n/7) + T(5n/7) + cn$$

因为:

$$\frac{1}{7} + \frac{5}{7} = \frac{6}{7} < 1$$

由 Problem Set 6 第 1(a) 的结论:

$$T(n) = O(n)$$

第 2(b) 问: $r = 3$ 时为什么不是 $O(n)$

一、分组为 3 时的结构

每组 3 个元素，排序后中位数是第 2 个。

设 pivot 是所有组中位数的中位数。

至少有一半组的中位数大于等于 pivot。

对这些组，每组只能保证有 2 个元素大于等于 pivot：

- 组内中位数；
- 组内最大值。

所以至少有：

$$2 \cdot \frac{1}{2} \cdot \frac{n}{3} = \frac{n}{3}$$

个元素大于等于 pivot。

同理，至少有 $\frac{n}{3}$ 个元素小于等于 pivot。

因此，分区后较大一侧最多只能保证是：

$$n - \frac{n}{3} = \frac{2n}{3}$$

递推式变成：

$$T(n) \leq T(n/3) + T(2n/3) + O(n)$$

参考答案也给出了类似递推关系，并指出这种情况下运行时间会快于线性增长，而不是 $O(n)$ 。

二、为什么这不是线性

看每一层递归的总代价。

第一层处理 n 个元素，代价 cn 。

下一层有两个递归规模：

$$n/3 \text{ 和 } 2n/3$$

它们加起来仍然是：

$$n/3 + 2n/3 = n$$

所以第二层总代价仍然大约是 cn 。

继续往下，每一层的总规模仍大约是 n ，而递归深度大约是 $O(\log n)$ 。

因此总时间是：

$O(n \log n)$

而不是 $O(n)$ 。

更准确地说，分组大小为 3 时，pivot 的质量不够好：它只能保证丢掉约 $n/3$ 的元素，而还需要递归处理 $n/3$ 个中位数来找 pivot，于是两个递归规模之和达到 n ，无法得到线性时间。

第 3 题：两个数据库中找到总体中位数

一、原题翻译

你想分析两个独立数据库中的一些很难获取的数据。

每个数据库包含 n 个数值，所以总共有 $2n$ 个数值。可以假设所有数值互不相同。

你想找到这 $2n$ 个数中的中位数。这里把中位数定义为第 n 小的元素。

但是，你访问这些数值的唯一方式是向数据库查询。

一次查询中，你可以向某一个数据库指定一个值 k ，该数据库会返回它内部第 k 小的数。

因为查询代价很高，所以希望你用尽可能少的查询找到中位数。

请给出一个算法，使用最多 $O(\log n)$ 次查询找到中位数。

二、问题等价形式

设两个数据库内部排序后分别是：

$A[1] < A[2] < \dots < A[n]$

$B[1] < B[2] < \dots < B[n]$

但是我们不能直接拿到数组，只能通过查询得到：

- 查询数据库 A 的第 k 小，得到 $A[k]$ ；
- 查询数据库 B 的第 k 小，得到 $B[k]$ 。

目标是找：

$A \cup B$ 中第 n 小的元素。

这是经典的 **两个有序数组找第 k 小元素** 问题，只是数组不能直接访问，需要通过查询访问。

三、核心观察

如果我们想找两个有序序列合并后的第 k 小元素，可以比较：

$A[i]$ 和 $B[j]$

其中：

$$i + j = k$$

如果 $A[i] < B[j]$

那么 $A[1], \dots, A[i]$ 都不可能是第 k 小以后的元素，它们一定在较小的一边，可以被排除。

原因是：

对于 $A[i]$ ，比它小的元素最多包括：

- A 中前 $i - 1$ 个；
- B 中最多前 $j - 1$ 个，因为 $B[j] > A[i]$ 。

总共最多：

$$(i - 1) + (j - 1) = k - 2$$

所以 $A[i]$ 以及 A 中更前面的元素都在第 k 小之前，可以丢弃。

如果 $A[i] > B[j]$

同理，可以丢弃：

$B[1], \dots, B[j]$

每次丢掉大约一半候选元素，所以只需 $O(\log n)$ 轮。

四、完整伪代码

下面给出更标准、易理解的“查第 k 小”算法。初始时令 $k = n$ ，即可求题目要求的中位数。

算法：两个数据库中找第 k 小元素

输入：

数据库 A ，数据库 B

每个数据库支持 $\text{QueryA}(t)$ 、 $\text{QueryB}(t)$

$\text{QueryA}(t)$ 返回 A 中第 t 小元素

$\text{QueryB}(t)$ 返回 B 中第 t 小元素

初始目标 $k = n$

输出：

$A \cup B$ 中第 k 小元素

变量:

a_start 表示 A 当前候选区间的起始排名
a_len 表示 A 当前候选区间长度
b_start 表示 B 当前候选区间的起始排名
b_len 表示 B 当前候选区间长度

初始化:

a_start = 1
a_len = n
b_start = 1
b_len = n
k = n

循环:

1. 如果 **a_len** == 0:
 返回 QueryB(**b_start** + $k - 1$)

2. 如果 **b_len** == 0:
 返回 QueryA(**a_start** + $k - 1$)

3. 如果 **k** == 1:
 x = QueryA(**a_start**)
 y = QueryB(**b_start**)
 返回 min(**x**, **y**)

4. 选择:
 i = min(**a_len**, floor($k/2$))
 j = $k - i$

如果 **j** > **b_len**:
 j = **b_len**
 i = $k - j$

5. 查询:
 x = QueryA(**a_start** + $i - 1$)
 y = QueryB(**b_start** + $j - 1$)

6. 如果 **x** < **y**:

 丢弃 A 当前候选区间的前 **i** 个元素

 a_start = **a_start** + **i**
 a_len = **a_len** - **i**
 k = $k - i$

否则:

丢弃 B 当前候选区间的前 j 个元素

```
b_start = b_start + j
b_len = b_len - j
k = k - j
```

7. 回到步骤 1。

五、为什么查询次数是 $O(\log n)$

每轮中，通常取：

$$i \approx k/2, \quad j \approx k/2$$

并且丢掉其中一边的约 $k/2$ 个元素。

所以 k 至少大约减半。

因此循环轮数是：

$$O(\log n)$$

每一轮只查询两个数：

- 一个来自数据库 A ；
- 一个来自数据库 B 。

所以总查询次数是：

$$O(\log n)$$

参考答案采用的是区间不变式写法，本质也是每轮把候选范围减半，因此总查询次数为 $O(\log n)$ 。

六、正确性证明

证明不变式：

每一轮开始时，目标元素一定是当前两个候选区间合并后的第 k 小元素。

初始时，两个候选区间就是完整数据库：

$$A[1..n] \text{ 和 } B[1..n]$$

目标是第 n 小，因此不变式成立。

情况 1: $x = A[a_{start} + i - 1] < y = B[b_{start} + j - 1]$

并且:

$$i + j = k$$

此时可以丢掉 A 当前候选区间的前 i 个元素。

为什么?

因为这些元素都不可能是当前候选集合中的第 k 小。

对于 x 来说, 比它小或等于它的候选元素最多只有:

$$i + (j - 1) = k - 1$$

因此 x 以及它之前的 A 元素都排在第 k 小之前。

丢掉这 i 个元素后, 原来的第 k 小就变成剩余元素中的第 $k - i$ 小。

所以更新:

$$k \leftarrow k - i$$

正确。

情况 2: $x > y$

对称地, 可以丢掉 B 当前候选区间的前 j 个元素, 并令:

$$k \leftarrow k - j$$

终止情况

如果某个候选区间为空, 那么答案就是另一个区间中的第 k 小。

如果 $k = 1$, 答案就是两个候选区间首元素中的较小者。

因此算法正确。

第 4 题: $n \times n$ 网格图中用 $O(n)$ 次探测找局部最小值

一、原题翻译

给定一个 $n \times n$ 网格图 G 。

所谓 $n \times n$ 网格图，就是 $n \times n$ 棋盘的邻接图。更精确地说，节点集合是所有有序对：

(i, j)

其中：

$$1 \leq i \leq n, \quad 1 \leq j \leq n$$

两个节点 (i, j) 和 (k, l) 之间有边，当且仅当：

$$|i - k| + |j - l| = 1$$

也就是说，上下左右相邻的格子之间连边。

每个节点 v 有一个实数标签 x_v ，并且所有标签互不相同。

如果节点 v 的标签小于所有与它相邻的节点标签，则称 v 是一个 **局部最小值** (local minimum)。

标签不是一次性给出的。对于每个节点 v ，你只能通过探测该节点来获得 x_v 。

请说明如何只用 $O(n)$ 次节点探测，就找到图 G 的一个局部最小值。

注意：图中一共有 n^2 个节点。你必须说明为什么过程只使用 $O(n)$ 次探测，并证明它一定能找到局部最小值。

二、为什么不能直接扫描全图

如果把所有 n^2 个节点都探测一遍，当然可以找到全局最小值，而全局最小值一定是局部最小值。

但这需要：

$$O(n^2)$$

次探测。

题目要求只用：

$$O(n)$$

次探测。

所以必须用分治法。

三、算法直觉

对一个 $n \times n$ 网格：

1. 探测中间行；
2. 探测中间列；
3. 再探测当前子网格的四条边界；
4. 在这些被探测的点中找到值最小的点 a ；
5. 检查 a 的所有相邻点；
6. 如果 a 比所有邻居都小，返回 a ；
7. 否则，存在一个邻居 b ，满足 $x_b < x_a$ ；
8. 递归进入包含 b 的那个象限。

参考答案中也是：探测四条边、中间行和中间列；取这些节点中的最小值点 a ；如果 a 是局部最小值则返回，否则递归进入包含较小邻居的子网格。

四、完整伪代码

为了方便叙述，设当前处理的子网格为：

$$R = [r_1..r_2] \times [c_1..c_2]$$

它大致是一个正方形。

算法：GRID-LOCAL-MINIMUM(R)

输入：

当前子网格 $R = [r_1..r_2] \times [c_1..c_2]$

输出：

原始网格中的一个局部最小值

步骤：

1. 如果 R 的规模很小，例如边长 ≤ 2 ：

探测 R 中所有节点及其原图中的邻居
直接返回一个局部最小值

2. 令：

$r_mid = \text{floor}((r_1 + r_2) / 2)$
 $c_mid = \text{floor}((c_1 + c_2) / 2)$

3. 探测以下节点：

子网格 R 的上边界：所有 (r_1, c) , $c_1 \leq c \leq c_2$

子网格 R 的下边界：所有 (r_2, c) , $c_1 \leq c \leq c_2$

子网格 R 的左边界：所有 (r, c_1) , $r_1 \leq r \leq r_2$

子网格 R 的右边界：所有 (r, c_2) , $r_1 \leq r \leq r_2$

中间行：所有 (r_mid, c) , $c_1 \leq c \leq c_2$

中间列：所有 (r, c_mid) , $r_1 \leq r \leq r_2$

4. 在这些已经探测的节点中，找到标签值最小的节点 a 。
5. 探测 a 在原始网格中的所有邻居：
上、下、左、右，最多 4 个。
6. 如果 a 的标签小于所有存在的邻居标签：
返回 a
7. 否则：
选择一个邻居 b ，使得 $x_b < x_a$ 。
8. 根据 b 所在位置，选择由中间行和中间列划分出的那个子网格 R' ：
如果 b 在左上象限：
 $R' =$ 左上子网格
如果 b 在右上象限：
 $R' =$ 右上子网格
如果 b 在左下象限：
 $R' =$ 左下子网格
如果 b 在右下象限：
 $R' =$ 右下子网格
9. 返回 $\text{GRID-LOCAL-MINIMUM}(R')$

初始调用：

$\text{GRID-LOCAL-MINIMUM}([1..n] \times [1..n])$

五、为什么每层只需要 $O(n)$ 次探测

在一个边长为 q 的子网格里：

- 四条边界共有 $O(q)$ 个节点；
- 中间行有 q 个节点；
- 中间列有 q 个节点；
- 检查 a 的邻居最多 4 个节点。

所以每层探测次数是：

$O(q)$

递归子网格边长大约减半：

$q \rightarrow q/2$

因此探测次数满足递推式：

$T(n) = O(n) + T(n/2)$

展开：

$$T(n) = O(n) + O(n/2) + O(n/4) + \dots$$

这是等比数列：

$$T(n) = O(n)$$

六、为什么算法正确

正确性证明分两部分。

1. 如果算法返回 a ，那么 a 真的是局部最小值

算法在返回 a 前，会探测 a 在原始网格中的所有邻居。

如果确认：

$$x_a < x_w$$

对所有邻居 w 都成立，那么根据局部最小值定义， a 就是局部最小值。

所以只要算法返回，返回结果一定正确。

2. 如果 a 不是局部最小值，为什么可以递归到较小邻居所在象限

如果 a 不是局部最小值，那么存在邻居 b ，满足：

$$x_b < x_a$$

由于 a 是当前探测集合中标签最小的节点，而探测集合包括边界和中间十字线，所以：

$$x_a$$

小于所有边界和中间十字线上的节点。

又因为：

$$x_b < x_a$$

所以：

$$x_b$$

更小于所有这些被探测的边界和中间十字线节点。

因此，从 b 开始，如果不断走向标签更小的邻居，这条下降路径不可能穿过当前探测过的边界或中间十字线。

原因是：边界和中间十字线上的点标签都比 b 大，而下降路径只能走向更小的点。

所以这条下降路径会一直留在 b 所在的那个象限内部。

由于网格有限，且所有标签互不相同，沿着严格下降的路径不可能无限走下去，最终一定停在某个节点 z 。

这个节点 z 没有更小的邻居，因此是局部最小值。

而且 z 位于 b 所在的子网格中。

所以递归进入 b 所在象限不会丢失所有局部最小值，算法仍然能找到一个局部最小值。

七、一个直观图景

你可以把探测的中间行、中间列、四条边界看成一圈“围栏”和一条“十字分割线”。

算法先在围栏和十字线上找最小点 a 。

如果 a 已经比所有邻居都小，就结束。

如果有更小邻居 b ，那么 b 一定在某个象限内部，并且比围栏和十字线都小。

从 b 沿着越来越小的方向走，不可能爬回围栏或十字线，因为那些点都更大。

所以那个象限内部一定藏着一个局部最小值。

于是我们只需要递归搜索那个象限。

总结表

题目	核心方法	关键结论
PS5 第 1 题	状态扩展 BFS	每个格子拆成“未拆墙/已拆墙”两个状态，时间 $O(hw)$
PS5 第 2 题	修改 Dijkstra	把加法松弛改成 $\max$ 松弛，时间 $O(mn)$
PS6 第 1 题	递推式归纳	$\alpha + \beta < 1$ 得 $O(n)$ ， $\alpha + \beta = 1$ 得 $O(n \log n)$
PS6 第 2 题	Median-of-Medians	分组 7：递归规模和 $< n$ ，线性；分组 3：递归规模和约等于 n ，非线性
PS6 第 3 题	二分式选择	两个数据库查第 n 小， $O(\log n)$ 次查询

题目	核心方法	关键结论
PS6 第 4 题	网格分治	探测边界和中间十字线，递归进一个象限，总探测 $O(n)$